Rockchip Developer FAQ Storage

ID: RK-PC-YF-133

Release Version: V2.3.0

Release Date: 2026-06-12

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document is intended to help developers understand storage-related issues and provides guidance on storage configuration and debugging.

Product Version

Chip Models	Kernel Version
All Products	--

Target Audience

This document (this guide) is mainly intended for the following engineers:

Technical Support Engineers

Software Development Engineers

Revision History

Version	Author	Date	Description
V1.0.0	Zhao Yifeng, Lin Dingqiang	2021-05-08	Initial version
V1.1.0	Lin Dingqiang	2021-07-06	Mainly added some UBIFS exception cases
V1.2.0	Lin Dingqiang	2021-10-29	Added more Debug examples
V2.0.0	Lin Dingqiang	2024-01-08	Added more Debug examples, optimized document hierarchy
V2.1.0	Lin Dingqiang	2024-07-26	Added more Debug examples
V2.2.0	Lin Dingqiang	2024-09-29	Added support for W25N01GV CONT_READ, SPI Nor AUTO_MERGE support explanation
V2.2.1	Lin Dingqiang	2025-01-06	Corrected layout errors
V2.3.0	Lin Dingqiang	2026-06-12	Fixed some typos

Table of Contents

Rockchip Developer FAQ Storage

1. Firmware Flashing Issues
    1.1 The Role of Loader in the Flashing Process
    1.2 Flashing Tool Error
    1.3 Unable to Boot from SLC Nand Image
        1.3.1 pin38 Functional Pin Enabled
    1.4 Boot hangs at the SPL stage after flashing the SPI NAND image
    1.5 How to Enter Maskrom Mode
    1.6 How to Build and Test the SPL Firmware Independently
    1.7 How to Obtain Flash Images or Debug Information
        1.7.1 Export Image
        1.7.2 Dump All
        1.7.3 Flash Stress Test
        1.7.4 Flash Erase All
2. File System Issues
    2.1 UBIFS Partition Was Not Fully Erased Before Upgrade
    2.2 UBIFS Image Incompatibility with Flash Specifications
    2.3 UBIFS Image Larger Than Partition
    2.4 Corruption at the End of the UBIFS Firmware
    2.5 Excessive UBIFS journal reservation
    2.6 Establishing UBI block with SquashFS Missing ZLIB Support
    2.7 Insufficient UBIFS Empty Partition Capacity Leads to Mount Failure After Image Creation
    2.8 Choosing Between LZO and ZLIB for UBIFS
    2.9 UBIFS Image Expansion Exceeds Partition Size
    2.10 UBIFS Upgrade Failure in Loader Mode with Modified GPT
    2.11 UBIFS SPI Nor Support
    2.12 Low-level Format Recovery Solution for UBIFS userdata Partition Issues
    2.13 Improper Naming of UBIFS vol_name Leading to Exceptions
    2.14 Long Boot Time with UBIFS SquashFS
    2.15 For partitions of the same size, a UBIFS image on a 4KB-page NAND device requires more filesystem overhead than on a 2KB-page NAND device
    2.16 Insufficient UBIFS df Command Space
    2.17 Slow Mount Speed of JFFS2 File System
    2.18 JFFS2 Partition Not Erased
    2.19 JFFS2 and SPI Nor 4K Erase Conflict
    2.20 YAFFS File System Not Recommended
    2.21 YAFFS2 File System Not Supported by RK Solution
    2.22 EXT File System Nand Solution Infeasibility
    2.23 EXT2 File System Support for SPI Nor Flash
    2.24 EXT2 Lacks Discard Feature Leading to Significant Degradation in Later Read/Write Speed
    2.25 SquashFS Compression Format and Its Efficiency
3. eMMC Issues
    3.1 Probabilistic Communication Errors in eMMC Kernel
    3.2 eMMC Firmware Upgrade Fails to Detect Storage Device or Verification Errors
    3.3 eMMC Damage or Firmware Loss After Prolonged Use
    3.4 eMMC Firmware Upgrade Stuck in Maskrom Mode
4. Flash Issues
    4.1 Flash Read/Write Speed
    4.2 Flash Write Protection Feature
    4.3 Incompatibility of Two Flash Devices Selected by Software
    4.4 Common Prints for Failed Flash Kernel Storage Initialization

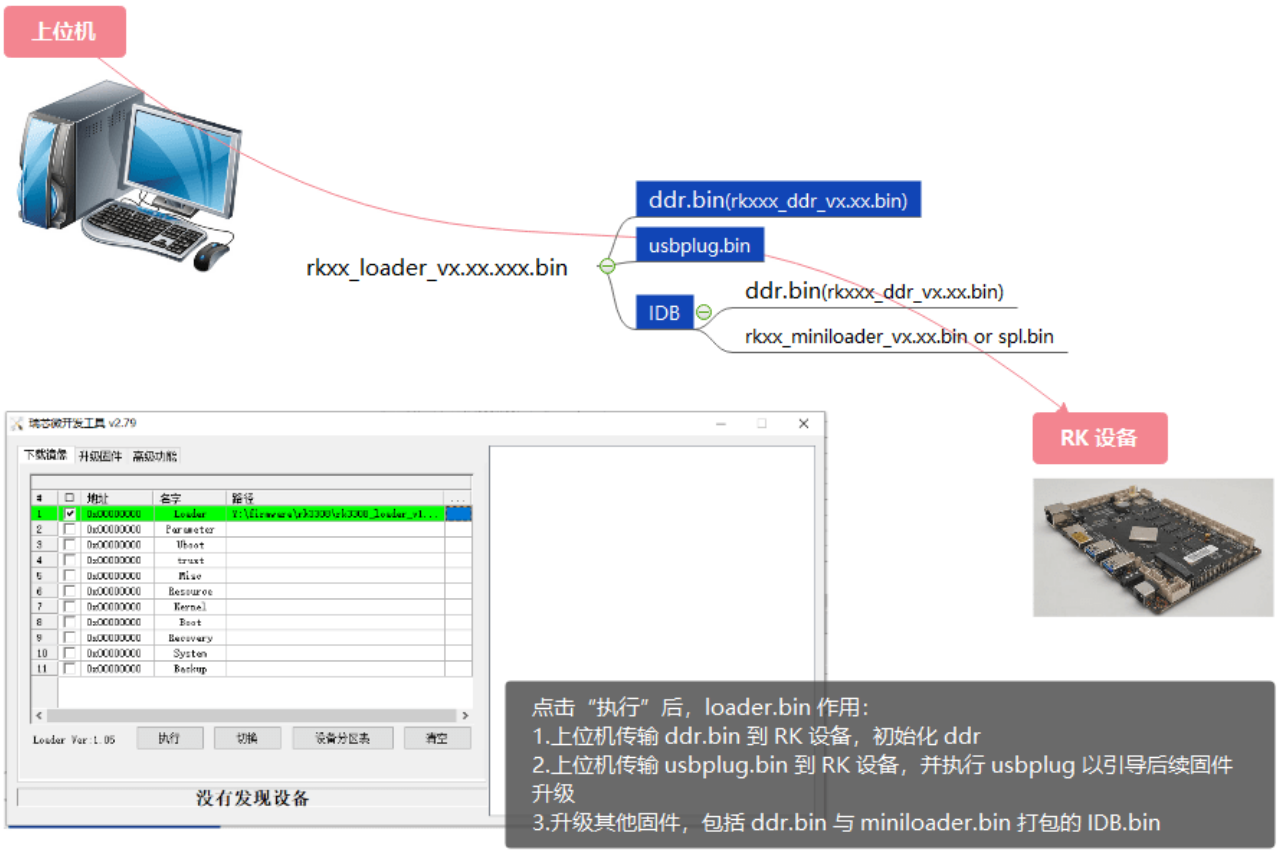
- 4.5 Ensuring Data Persistence in Flash Storage Device Nodes
- 4.6 Flash Material Replacement Instructions
- 4.7 Troubleshooting Flash TFTP Firmware Download and Burning Anomalies
- 4.8 Flash uboot Phase MTD Interface Comparison with MTD_BLOCK Interface
- 4.9 Flash Cold Temperature Test Anomaly
- 4.10 Flash Development Board Maskrom Button Proximity to Touch Affects Data Transmission

5. Flash Controller Issues
   - 5.1 Controller FSPI Clock Duty Cycle Below 50%
   - 5.2 Controller FSPI Non-General SPI Controller, Only Supports SPI Memory Protocol
   - 5.3 Controller FSPI Establishment and Hold Time Not Measured
   - 5.4 Does the FSPI controller support connecting different devices to separate chip-select lines?
6. NAND Issues
   - 6.1 PX30 Nand Device Modified to rkflash Storage Driver
   - 6.2 Nand uboot Custom Read/Write Behavior Standards
   - 6.3 NAND SFTL Space Planning
   - 6.4 NAND Flash Pre-Burning Does Not Allow Combining Separate Images for One-Time Burning
   - 6.5 NAND Open Source Framework Bad Block Table Management
7. PP SLC NAND Issues
   - 7.1 PX30 SLC Nand IDB Creation Issues
   - 7.2 Analysis of SLC NAND devices from Original Manufacturers Reveals Continuous Large Numbers of Bad Blocks
   - 7.3 SLC NAND Flash Programmer Does Not Support Master Device Images
   - 7.4 SLC NAND Programmer Burning Images with Valid Data and Redundant OOB Data
   - 7.5 SLC NAND SDK Outputs Valid Data Image (Excluding Redundant OOB Data)
   - 7.6 SLC NAND Software ECC Layout
   - 7.7 SLC NAND rkflash Solution Kernel Phase Read and Write Speed Optimization
   - 7.8 SLC NAND mtd Solution SPL Phase Read Transfer Rate Optimization
   - 7.9 SLC NAND Excessive Bad Blocks at the End MTD Solution Device Failure to Operate Normally
8. SPI Nand Common Issues
   - 8.1 SPI Nand Flash Devices Plane Select Material Compatibility Issues
   - 8.2 1V8 SPI Nand Flash Devices Selection
   - 8.3 SPI Nand Flash HYF1GQ4UTXCAE Anomaly Power Down Issue
   - 8.4 Preliminary Research on SPI Nand Software Soft ECC Solution
   - 8.5 SPI Nand Software BBT Store in Flash Protection Mechanism
   - 8.6 The SPI NAND programmer supports programming the master device, but does not support exporting the entire image in a single operation
   - 8.7 SPI Nand SDK Outputs Valid Data Image Without Redundant OOB Data
   - 8.8 SPI Nand rkflash Kernel Phase Read/Write Speed Optimization
   - 8.9 SPI Nand mtd Solution Kernel Phase Flash Information and Bad Block Information
   - 8.10 SPI Nand mtd Solution Kernel Phase mtd Write Failure
   - 8.11 SPI Nand mtd Solution Root Filesystem Auto-mount Compatibility with eMMC Solution
   - 8.12 SPI Nand mtd Solution for env Partition Table Upgrade Compatibility with 4K page-size NAND devices using idb Image
   - 8.13 SPI Nand Fast Boot Solution Kernel Failure to Establish Bad Block Table
   - 8.14 SPI Nand Custom MTD Upgrade Solution Encounters Anomalies with Bad Block Partitions
   - 8.15 SPI Nand Continuous Read Support and Quick Boot Adaptation
     - 8.15.1 Introduction
     - 8.15.2 Configuration
9. SPI NOR Issues
   - 9.1 SPI Nor Flash 1V8 256Mbits and Higher Capacity Material Selection
   - 9.2 SPI Nor Flash Solutions Compared with Kernel 4.4 and Earlier Versions of the MTD Open Source Framework
   - 9.3 SPI Nor mtd Solution u-boot Down sf Command and Related Interface Implementation
   - 9.4 SPI Nor mtd Registration of mtdparts Takes Long Time

9.5 SPI Nor mtd Solution: Slow Erasure Speed in U-Boot
9.6 SPI Nor Single-Line and Quad-Line Transfer Performance Differences
9.7 SPI Nor High-Speed Copying Anomaly
9.8 SPI Nor AUTO_MERGE Technology
9.8.1 Introduction
9.8.2 Limitations
9.8.3 Configuration
9.8.4 CSN1 Configuration
10. Tool Issues
10.1 Creating GPT Image with upgrade_tool
10.2 Boot Merger Unpack Loader
10.3 Android Upgrade Utility LoaderToDDR Features
10.4 Introduction to the idb_bootconfig Tool

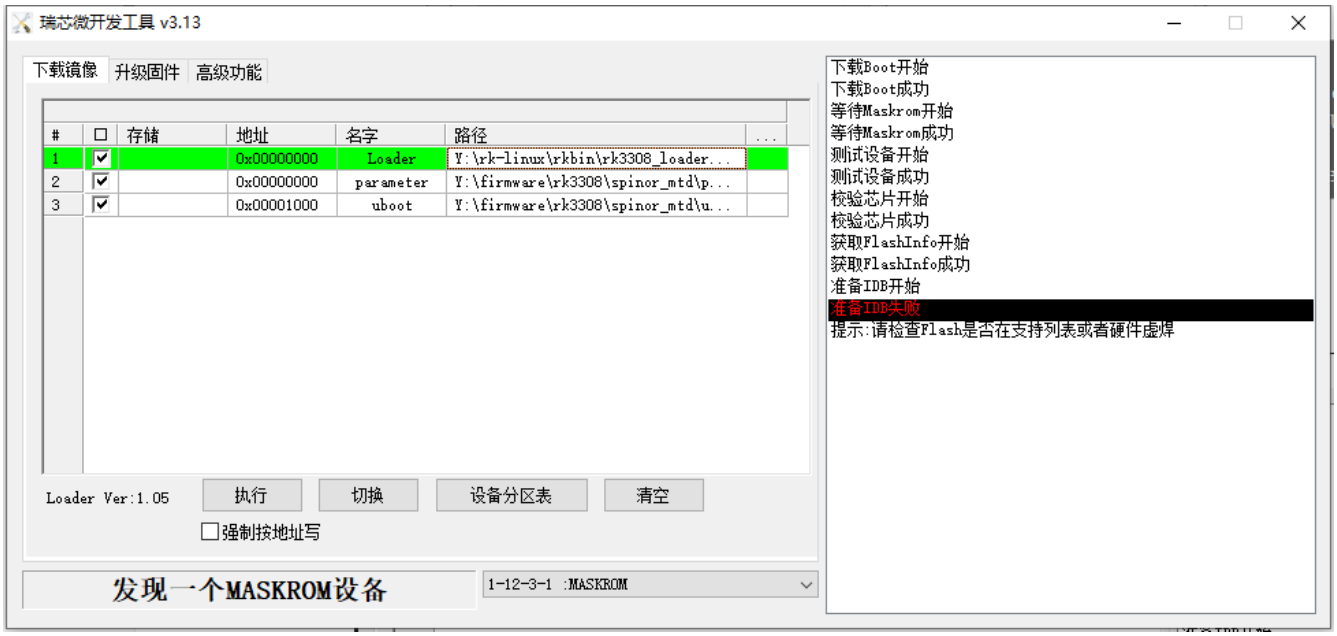
1. Firmware Flashing Issues

1.1 The Role of Loader in the Flashing Process



1.2 Flashing Tool Error

Tool Interface:



Tool log:

```
18:05:40 125    Layer<1-1-2-3-1>: Download Boot successful
18:05:40 127    Layer<1-1-2-3-1>: Waiting for Maskrom to start
18:05:40 552    Layer<1-1-2-3-1>: Waiting for Maskrom successful
18:05:40 557    Layer<1-1-2-3-1>: Testing device started
18:05:40 572    Layer<1-1-2-3-1>: Testing device successful
18:05:40 583    Layer<1-1-2-3-1>: Verifying chip started
18:05:40 592    Layer<1-1-2-3-1>: Verifying chip successful
18:05:40 600    Layer<1-1-2-3-1>: Obtaining FlashInfo started
18:05:40 606    <LAYER 1-12-3-1> INFO:FlashInfo: 00 00 00 00 00 04 04 00 28 00 00
18:05:40 615    <LAYER 1-12-3-1> INFO:GetFlashInfo-->Emmc storage.
18:05:40 626    Layer<1-1-2-3-1>: Obtaining FlashInfo successful
18:05:40 640    Layer<1-1-2-3-1>: Preparing IDB started
18:05:40 645    <LAYER 1-12-3-1> ERROR:PrepareIDB-->No Found 1st Flash CS
18:05:40 663    Error:Layer<1-1-2-3-1>:Preparing IDB failed
18:05:40 686    Layer<1-12-3-1>: RunProc is ending, ret=0
```

Device UART Serial Print:

```
ddr.bin log:     DDR Version V1.30 20191125
ddr.bin log:     REG2C: 0x00000031, 0x00000031
ddr.bin log:     In
ddr.bin log:     451MHz
ddr.bin log:     DDR2
ddr.bin log:     Col=10 Bank=2 Row=13 Size=64MB
ddr.bin log:     msch:0
ddr.bin log:     OUT
usbplug log:     Boot1 Release Time: Dec  7 2020 11:25:17, version: 1.25
usbplug log:     chip_id:000000,0
usbplug log:     DPLL = 1300 MHz
usbplug log:     ...nandc_flash_init enter...
```

```
usbplug log:     No.1 FLASH ID:ff ff ff ff ff ff                          /* flash id all
FF, SLC Nand not detected */
usbplug log:     DPLL = 1300 MHz
usbplug log:     sfc nor id: ff ff ff                                     /* flash id all
FF, SPI Nor not detected */
usbplug log:     DPLL = 1300 MHz
usbplug log:     sfc_nand id: ff ff ff                                    /* flash id all
FF, SPI Nand not detected */
usbplug log:     UsbBoot ...11943
usbplug log:     powerOn 664713
```

Problem Analysis:

1. If there is an anomaly in the ddr.bin log, please contact the DDR engineer.
2. If the storage used is parallel SLC Nand, SPI Nand, or SPI Nor, and the flash ID is all FF during the upgrade process, it is mostly due to abnormal flash vcc power supply, IO voltage level anomaly, or soldering anomaly.
3. If the Flash ID is detected correctly but "The device not support yet!" is printed, and the corresponding platform is listed as supported in the "RK SpiNor and SLC Nand Support List", the current firmware is outdated and does not include support for the device. Please update the storage driver before proceeding: Redmine -> FAE Engineering -> Documents -> Nand flash support list -> Memory Patch Link
4. If it still cannot be downloaded correctly after synchronization, please contact the RK storage group engineer.

1.3 Unable to Boot from SLC Nand Image

1.3.1 pin38 Functional Pin Enabled

Common Burn-in Log:

```
prog read s error: = 7fc00 1fff0e0 ffffffff
prog read d error: = 7fc00 0 ffffffff
phyBlk = 0x1ff die = 0 block_in_die = 0x1ff 0x80000000
prog read s error: = 7f800 1fef0e0 ffffffff
prog read d error: = 7f800 0 ffffffff
phyBlk = 0x1fe die = 0 block_in_die = 0x1fe 0xc0000000
prog read s error: = 7f400 1fdf0e0 ffffffff
prog read d error: = 7f400 0 ffffffff
```

Please confirm if flash pin38 is a functional pin, and if it is configured to remove the corresponding enable level:

- Lock#, typically low active, with internal weak pull-up, please leave the pin floating or externally add a pull-up resistor, refer to the manual for details
- PT, typically high active, with internal weak pull-down, please leave the pin floating or externally add a pull-down resistor, refer to the manual for details
- NC, no functional pin, please leave it floating

1.4 Boot hangs at the SPL stage after flashing the SPI NAND image

If the following log is present, it indicates that the SDK used by the user does not include software support for the corresponding spinand. Please obtain the corresponding storage patch, which can be found under "Redmine -> FAE Project -> Documentation -> Nand Flash Support List".

```
DDR Version V1.04 20201030
DDR, 328MHz
BW=32 Col=10 Bk=8 CS0 Row=15 CS=1 Die BW=16 Size=1024MB
change to 328MHz
change to 528MHz
change to 784MHz
change to 924MHz(final freq)
out
U-Boot SPL board init
U-Boot SPL 2017.09.g2d25c32e07-20201118 #huke (Feb 26 2021 - 13:51:03)
unknown raw ID phN    # This log indicates that the spl driver does not support this memory device, rebuild spl is needed, and verify and update the corresponding patches for uboot, kernel
unrecognized JEDEC id bytes: ff, 98, ed
Trying to boot from MMC1
Card did not respond to voltage select!
mmc_init: -95, time 9
spl: mmc init failed with error: -95
SPL: failed to boot from all boot devices
### ERROR ### Please RESET the board ###
# Reset the board to bootrom #
```

1.5 How to Enter Maskrom Mode

1. For devices with a maskrom button: Press and hold the reset button, then press the maskrom button, release the reset button (you will see the tool display maskrom), then release the maskrom button (the order is crucial to avoid exceptions);
2. For devices without a maskrom button: The maskrom button is replaced by "shorting the flash data line IO0 to ground".

1.6 How to Build and Test the SPL Firmware Independently

Changing Compilation Method

The SDK uses pre-compiled SPL firmware, which is located in the rkbin/bin/xx/ directory. This is the loader firmware used for packaging, which is not the latest compiled SPL firmware. You can refer to the following commands to use self-compiled SPL firmware:

```
cd uboot
./make.sh rv1126
./make.sh spl-s
```

The output files in the uboot path are: rv1126_spl_loader_vxxx.xxx.bin and uboot.img. Please use this loader.bin and uboot.img.

How to Confirm Successful SPL Update

"U-Boot SPL 2017.09-gf899931b5f-201209 #ldq (Dec 10 2020 - 15:34:13)"

You can confirm whether the SPL has been updated to the self-compiled version through the SPL log.

How to Replace SDK SPL with Self-Compiled SPL

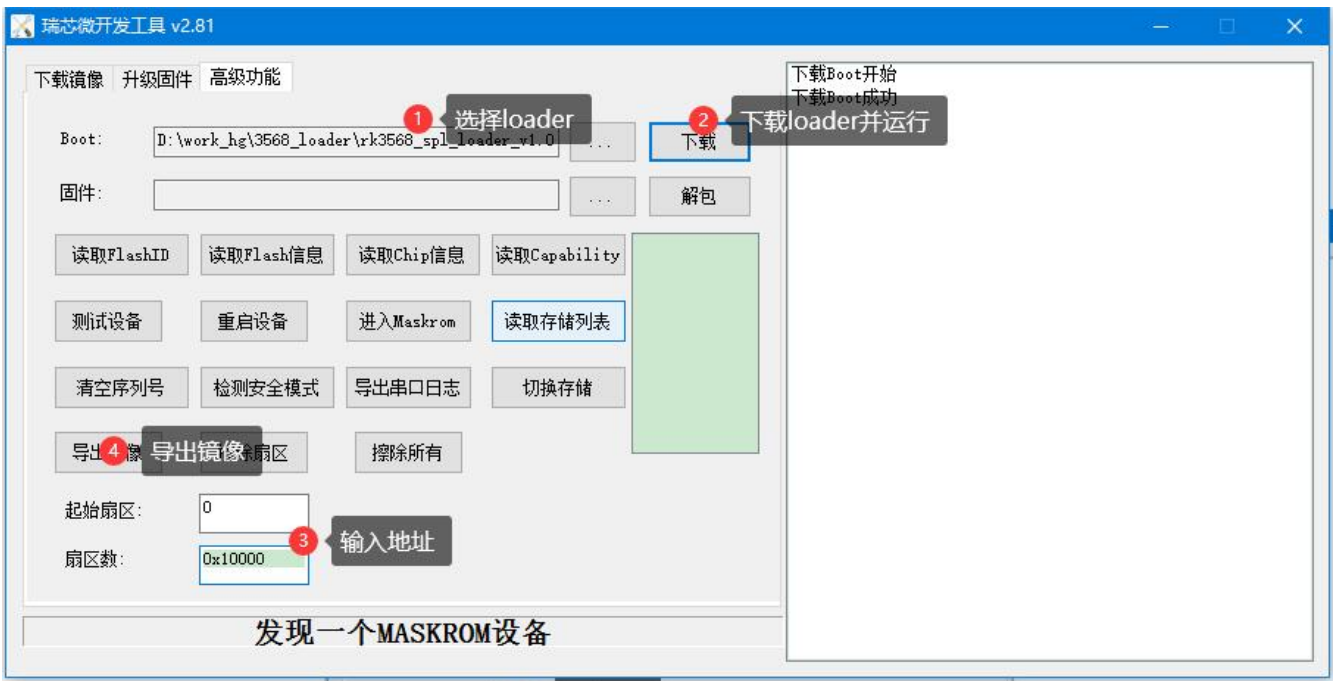
Replace the compiled uboot/spl/u-boot-spl.bin with ./rkbin/bin/xx/xxxx_spl_xxxx.bin.

1.7 How to Obtain Flash Images or Debug Information

1.7.1 Export Image

Android tool

Exporting firmware in Maskrom mode:



Exporting firmware in Loader mode:

No need to download the loader; the tool directly inputs the address to export the image.

Linux upgrade tool & Android_Console_Tool

Exporting firmware in Maskrom mode:

```
db MiniloadAll.bin
rl 0x0 0x10000 Exportimage.bin
```

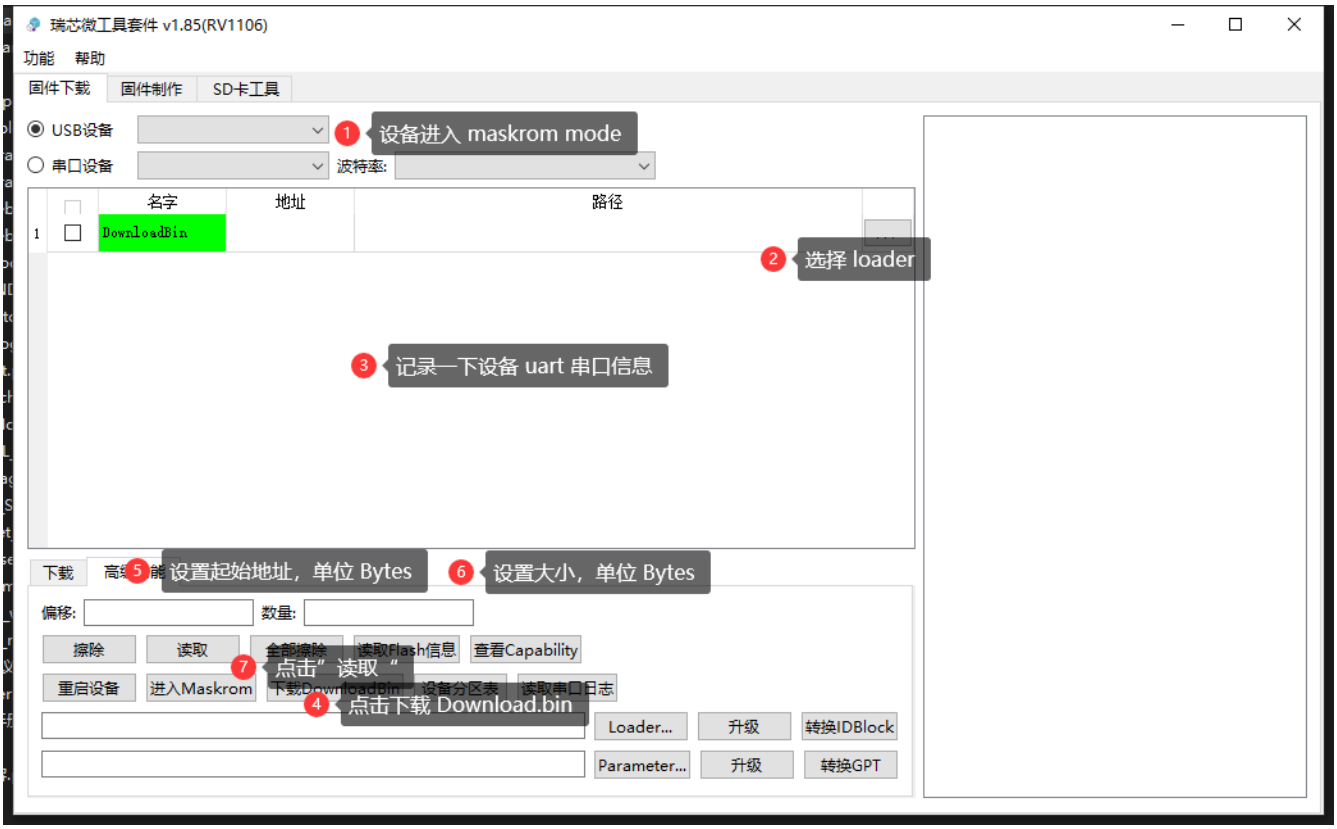
Exporting firmware in Loader mode:

```
rl 0x0 0x10000 Exportimage.bin
```

Note:

- The sector address unit is 512B per sector

toolkit



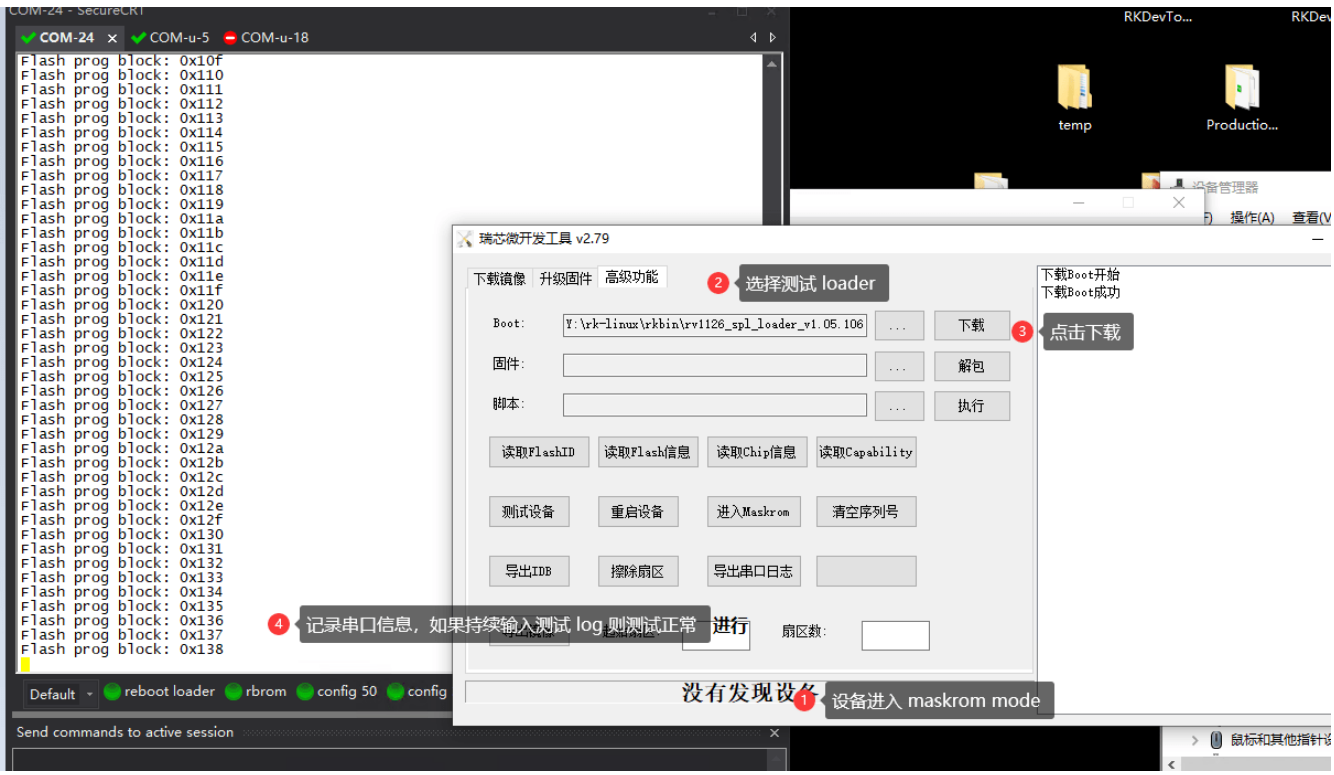
1.7.2 Dump All

A special loader with the suffix `dump_all.bin` is used to export specific flash information for RK ftl.



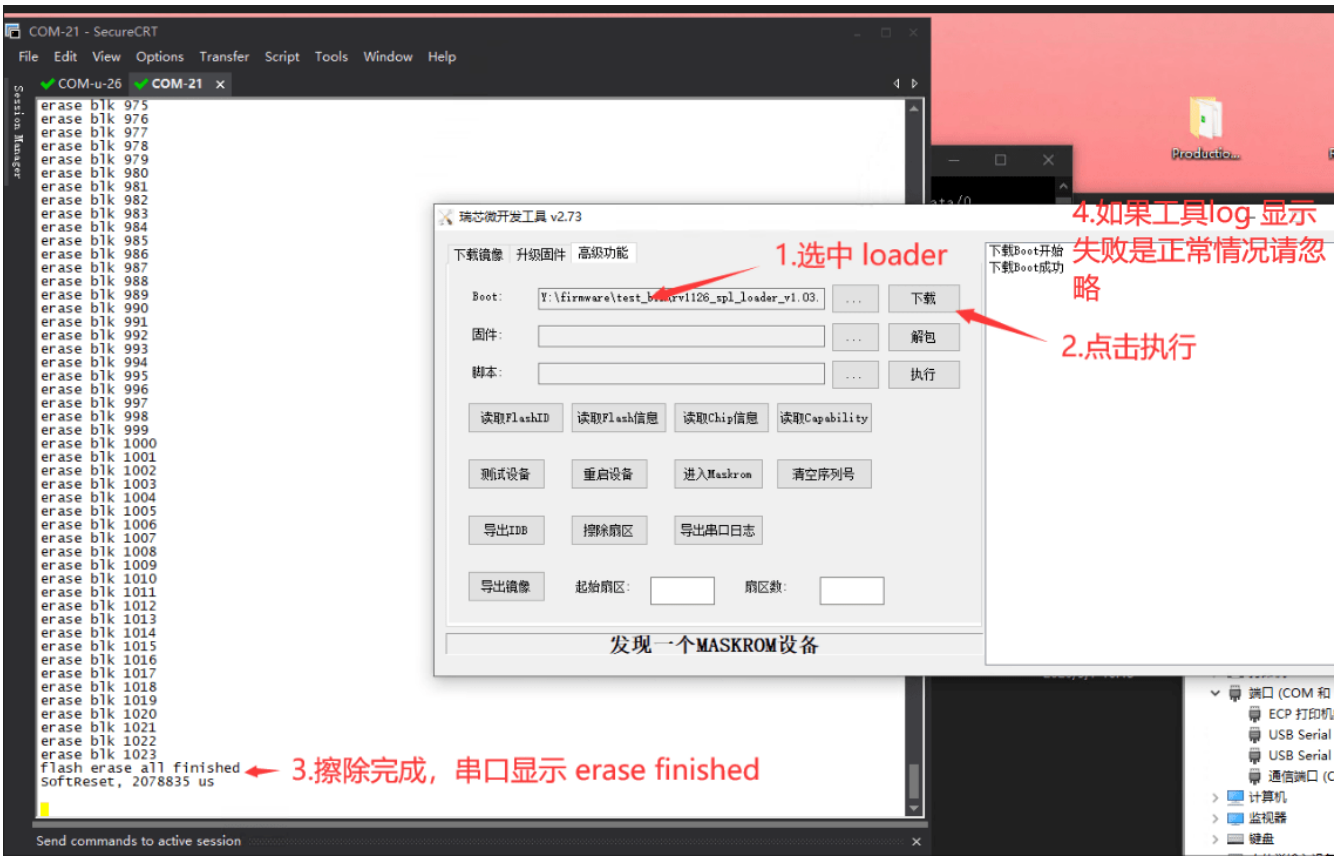
1.7.3 Flash Stress Test

The special loader suffix is stress_test.bin, which performs simple read, write, and erase stress testing on the flash.



1.7.4 Flash Erase All

The special loader suffix `erase_all.bin` is used to completely erase the flash data.



2. File System Issues

2.1 UBIFS Partition Was Not Fully Erased Before Upgrade

When upgrading an image using UBIFS, it is required to erase the entire partition first or use the ubiupdatevol tool for the upgrade (recommended). Common error messages are as follows:

```
...
[    4.452318] ubi0: attaching mtd5
[    4.685258] ubi0 error: ubi_attach: bad image sequence number 1251817256 in PEB 543,
expected 1051610977
[    4.685317] Erase counter header dump:
[    4.685334]  magic          0x55424923
[    4.685348]  version        1
[    4.685361]  ec             1
[    4.685374]  vid_hdr_offset 2048
[    4.685386]  data_offset    4096
[    4.685400]  image_seq      1251817256
[    4.685413]  hdr_crc        0x88225c8a
[    4.685426] erase counter header hexdump:
[    4.685753] ubi0 error: ubi_attach_mtd_dev: failed to attach mtd5, error -22
[    4.685814] UBI error: cannot attach mtd5
```

```
...
[    4.695772] ALSA device list:
[    4.695796]   #7: Loopback 1
[    4.696063] VFS: Cannot open root device "ubi0:rootfs" or unknown-block(0,0): error -19
[    4.696091] Please append a correct "root=" boot option; here are the available
partitions:
...
```

Recommended Tool Versions:

- Windows rkdevtool v2.79 and above
- Linux_Upgrade_Tool_v1.59 and above

2.2 UBIFS Image Incompatibility with Flash Specifications

Ensure that the `-e` and `-m` options used in the `mkfs` command when creating the image are consistent with the flash-related information. If they are inconsistent, the system will be unable to locate UBI-related information after startup, typically resulting in the following errors:

case 1:

```
...
[    1.319729] ubi0: attaching mtd3
[    1.549286] ubi0: scanning is finished
[    1.549326] ubi0 error: ubi_read_volume_table: the layout volume was not found
[    1.549464] ubi0 error: ubi_attach_mtd_dev: failed to attach mtd3, error -22
[    1.549499] UBI error: cannot attach mtd3
...
[    1.563848]   #7: Loopback 1
[    1.564173] UBIFS error (pid: 1): cannot open "ubi0:rootfs", error -19
[    1.564183] VFS: Cannot open root device "ubi0:rootfs" or unknown-block(0,0): error -19
...
```

case 2: Actual page size 4KB, image page size 2KB

```
[    0.517603] ubi0 error: validate_ec_hdr: bad VID header offset 2048, expected 4096
[    0.517618] ubi0 error: validate_ec_hdr: bad EC header
[    0.517626] Erase counter header dump:
[    0.517634]  magic          0x55424923
[    0.517639]  version        1
[    0.517645]  ec             0
[    0.517650]  vid_hdr_offset 2048
[    0.517657]  data_offset    4096
[    0.517664]  image_seq      2053699779
[    0.517672]  hdr_crc        0x2416c7f2
[    0.517679] erase counter header hexdump:
[    0.517701] CPU: 0 PID: 1 Comm: swapper/0 Not tainted 4.19.111 #2
```

case 3: Actual page size 2KB, image page size 4KB

```
[    1.678775] ubi0: attaching mtd6
[    1.679040] ubi0 error: validate_ec_hdr: bad VID header offset 4096, expected 2048
[    1.679054] ubi0 error: validate_ec_hdr: bad EC header
[    1.679068] Erase counter header dump:
[    1.679079]  magic          0x55424923
[    1.679090]  version        1
[    1.679101]  ec             0
[    1.679110]  vid_hdr_offset 4096
[    1.679120]  data_offset    8192
[    1.679131]  image_seq      362848614
[    1.679141]  hdr_crc        0x9438b345
[    1.679157] erase counter header hexdump:
[    1.679195] CPU: 0 PID: 1 Comm: swapper/0 Not tainted 4.19.172 #8
[    1.679207] Hardware name: Rockchip RK3566 For 360 X200 (DT)
[    1.679220] Call trace:
[    1.679238]  dump_backtrace+0x0/0x178
```

2.3 UBIFS Image Larger Than Partition

Reduce the image size or increase the partition size, typically the error message is as follows:

```
[    2.121208] ubi0 error: vtbl_check: too large reserved_pebs 824, good PEBs 767
[    2.121295] ubi0 error: vtbl_check: volume table check failed: record 0, error 9
[    2.121338] Volume table record 0 dump:
[    2.121375]  reserved_pebs   824
[    2.121412]  alignment       1
[    2.121449]  data_pad        0
[    2.121529]  vol_type        1
[    2.121568]  upd_marker      0
[    2.121605]  name_len        6
[    2.121642]  name            rootfs
[    2.121680]  crc             0x9ab0c6a3
[    2.122162] ubi0 error: ubi_attach_mtd_dev: failed to attach mtd3, error -22
[    2.122278] UBI error: cannot attach mtd3
```

2.4 Corruption at the End of the UBIFS Firmware

This issue requires further analysis. The error messages are typically as follows:

```
[    1.867789] ALSA device list:
[    1.867830]   #7: Loopback 1
[    1.916907] UBIFS (ubi0:0): UBIFS: mounted UBI device 0, volume 0, name "rootfs", R/O mode
[    1.917010] UBIFS (ubi0:0): LEB size: 126976 bytes (124 KiB), min./max. I/O unit sizes:
2048 bytes/2048 bytes
[    1.917061] UBIFS (ubi0:0): FS size: 103231488 bytes (98 MiB, 813 LEBs), journal size
9023488 bytes (8 MiB, 72 LEBs)
[    1.917106] UBIFS (ubi0:0): reserved for root: 0 bytes (0 KiB)
[    1.917155] UBIFS (ubi0:0): media format: w4/r0 (latest is w5/r0), UUID 1C1DDBB4-77E6-
4AA9-855F-ED8F10EB5916, small LPT model
```

2.5 Excessive UBIFS journal reservation

UBIFS is a journaling file system. If the journal retention is too large and the partition is small, it may not be able to mount normally.

Consider adjusting the parameters:

```
-j, --jrn-size=SIZE       journal size
```

Typical error messages are as follows:

```
[root@RV1126_RV1109:/]# mount -t ubifs /dev/ubi2_0 /record/
[   65.788339] UBIFS error (ubi2:0 pid 890): ubifs_read_superblock: mount: mounting
/dev/ubi2_0 on /record/ failed: Invalid argument
too large journal size (8388608 bytes), only 3555328 bytes available in the main area
[   65.788394] UBIFS error (ubi2:0 pid 890): ubifs_read_superblock: bad superblock, error 1
[   65.788413]  magic          0x61018311
[   65.788434]  crc            0x663d9574
[   65.788455]  node_type      6 (superblock node)
[   65.788475]  group_type     0 (no node group)
[   65.788493]  sqnum          1
[   65.788512]  len            4096
[   65.788531]  key_hash       0 (R5)
[   65.788620]  key_fmt        0 (simple)
[   65.788639]  flags          0x4
[   65.788660]  big_lpt        0
[   65.788684]  space_fixup    1
[   65.788706]  min_io_size    2048
[   65.788728]  leb_size       126976
[   65.788755]  leb_cnt        39
[   65.788773]  max_leb_cnt    2048
[   65.788786]  max_bud_bytes  8388608
[   65.788799]  log_lebs       5
[   65.788812]  lpt_lebs       2
[   65.788826]  orph_lebs      1
[   65.788838]  jhead_cnt      1
[   65.788851]  fanout         8
[   65.788864]  lsave_cnt      256
[   65.788877]  default_compr  1
```

```
[   65.788890]  rp_size        0
[   65.788905]  rp_uid         0
[   65.788918]  rp_gid         0
[   65.788931]  fmt_version    4
[   65.788945]  time_gran      1000000000
[   65.788961]  UUID           DBC08BA0-15FE-49BD-BC47-8A9200F1D625
```

2.6 Establishing UBI block with SquashFS Missing ZLIB Support

Enable the macro CONFIG_SQUASHFS_ZLIB, and you will typically encounter the following error:

```
[    4.564088] rockchip-mipi-dphy-rx: No link between dphy and sensor
[    4.564102] rkisp0: update sensor failed
[    4.564181] ALSA device list:
[    4.564191]   #0: rockchip,rk809-codec
[    4.564200]   #7: Loopback 1
[    4.566057] squashfs: SQUASHFS error: Filesystem uses "zlib" compression. This is not
supported
[    4.566158] List of all partitions:
[    4.566179] fd00           31792 ubiblock0_0
[    4.566181]  (driver?)
[    4.566196] No filesystem could mount root, tried:
[    4.566197]  squashfs
[    4.566206]
[    4.566225] Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-
block(253,0)
[    4.570008] CPU: 3 PID: 1 Comm: swapper/0 Not tainted 4.19.149 #3
[    4.570547] Hardware name: Generic DT based system
[    4.570993] [<b010f47c>] (unwind_backtrace) from [<b010b9a8>] (show_stack+0x10/0x14)
[    4.571677] [<b010b9a8>] (show_stack) from [<b0827b68>] (dump_stack+0x90/0xa4)
[    4.572317] [<b0827b68>] (dump_stack) from [<b01264fc>] (panic+0x114/0x2a0)
[    4.572962] [<b01264fc>] (panic) from [<b0c013a8>] (mount_block_root+0x2a4/0x2f4)
[    4.573629] [<b0c013a8>] (mount_block_root) from [<b0c015d0>] (prepare_namespace+0x150/0x1
```

2.7 Insufficient UBIFS Empty Partition Capacity Leads to Mount Failure After Image Creation

UBIFS Minimum Partition:

```
Minimum block num = 4 (reserved) + B + 17  /* B - flash blocks reserved for bad block
replacement, related to the ubiattach -b parameter */
```

It can be determined by printing the log during ubiattach, for example:

```
ubi4: available PEBs: 7, total reserved PEBs: 24, PEBs reserved for bad PEB handling: 20   /*
B = 20 */
```

If the partition available PEBs + total reserved PEBs < Minimum block num, an error will be reported during mounting:

```
mount: mounting /dev/ubi4_0 on userdata failed: Invalid argument
```

2.8 Choosing Between LZO and ZLIB for UBIFS

The RK SDK defaults to using LZO compression, which can compress a specific case of 103M to 80M. ZLIB offers a similar compression ratio. If a higher compression ratio is required, SquashFS can be utilized.

2.9 UBIFS Image Expansion Exceeds Partition Size

When creating UBIFS images, the autoresize feature is typically used. If the image size is specified in the configuration file corresponding to the following command, it is likely that the image expansion will exceed the partition size:

```
ubinize -o ubi.img -m 2048 -p 128kiB -O 2048 ubinize.cfg

#ubinize.cfg
[ubifs]
  mode=ubi
  image=ubifs.img
  vol_id=0
  vol_size=192MB                // No need to specify size, use the autoresize feature
  vol_type=dynamic
  vol_name=rootfs
  vol_flags=autoresize
```

The typical error message is as follows:

```
[    2.345243] ubi0: attaching mtd2
[    2.415631] ubi0: scanning is finished
[    2.419742] ubi0 error: vtbl_check: too large reserved_pebs 1586, good PEBs 699
[    2.419785] ubi0 error: vtbl_check: volume table check failed: record 0, error 9
[    2.419792] Volume table record 0 dump:
[    2.419798]  reserved_pebs   1586
[    2.419803]  alignment       1
[    2.419808]  data_pad        0
[    2.419813]  vol_type        1
[    2.419818]  upd_marker      0
[    2.419823]  name_len        6
[    2.419829]  name            rootfs
[    2.419835]  crc             0x23e79a61
[    2.420000] ubi0 error: ubi_attach_mtd_dev: failed to attach mtd2, error -22
[    2.420024] UBI error: cannot attach mtd2
```

2.10 UBIFS Upgrade Failure in Loader Mode with Modified GPT

In the RK SDK mtd boot scheme, if an upgrade of "firmware with modified GPT" is performed in loader mode, it will cause the target firmware to be upgraded on the old bad block mapping relationship generated at u-boot startup. Therefore, loader mode only supports firmware upgrades without GPT modifications. Otherwise, please switch to maskrom mode for upgrades. If an incorrect upgrade is performed, the following logs are typically observed:

case 1:

```
[    0.659432] ubi0: attaching mtd3
[    1.264467] ubi0: scanning is finished
[    1.271151] ubi0 warning: ubi_read_volume_table: static volume 0 misses 1 LEBs - corrupted
[    1.302342] ubi0: volume 0 ("rootfs") re-sized from 613 to 636 LEBs
[    1.303492] ubi0: attached mtd3 (name "rootfs", size 85 MiB)
[    1.303526] ubi0: PEB size: 131072 bytes (128 KiB), LEB size: 126976 bytes
[    1.303533] ubi0: min./max. I/O unit sizes: 2048/2048, sub-page size 2048
[    1.303540] ubi0: VID header offset: 2048 (aligned 2048), data offset: 4096
[    1.303547] ubi0: good PEBs: 677, bad PEBs: 3, corrupted PEBs: 0
[    1.303559] ubi0: user volume: 1, internal volumes: 1, max. volumes count: 128
[    1.303568] ubi0: max/mean erase counter: 1/0, WL threshold: 4096, image sequence number:
980761742
[    1.303578] ubi0: available PEBs: 0, total reserved PEBs: 677, PEBs reserved for bad PEB
handling: 37
[    1.303599] ubi0: background thread "ubi_bgt0d" started, PID 80
```

case 2:

```
[11:34:08:931][    0.661124] ubi0: attaching mtd6
[11:34:08:931][    0.662524] ubi0 error: ubi_attach: bad image sequence number 67638577 in
PEB 1, expected 980761742
[11:34:08:931][    0.662571] Erase counter header dump:
[11:34:08:931][    0.662578]  magic          0x55424923
[11:34:08:931][    0.662584]  version        1
[11:34:08:931][    0.662590]  ec             0
[11:34:08:931][    0.662595]  vid_hdr_offset 2048
[11:34:08:931][    0.662601]  data_offset    4096
[11:34:08:931][    0.662606]  image_seq      67638577
[11:34:08:947][    0.662611]  hdr_crc        0x3a381cee
[11:34:08:947][    0.662616] erase counter header hexdump:
[11:34:08:949][    0.662645] ubi0 error: ubi_attach_mtd_dev: failed to attach mtd6, error -22
[11:34:08:949][    0.662675] UBI error: cannot attach mtd6
[11:34:08:949][    0.662691] UBI: block: can't open volume on ubi0_-1, err=-19
```

2.11 UBIFS SPI Nor Support

Emphasis:

SPI Nor is recommended to use JFFS2 as the choice for a writable file system.

Kernel Configuration:

```
CONFIG_MTD_SPI_NOR_USE_4K_SECTORS=n
```

Image Creation:

```
mkfs.ubifs -r data/ -m 1 -e 65408 -c 62 userdata.ubifs
ubinize -o ubi.img -m 1 -p 64KiB -O 64 ubinize_userdata.cfg

$cat ubinize_userdata.cfg
[userdata-volume]
mode=ubi
image=userdata.ubifs
vol_id=0
vol_size=2MiB
vol_type=dynamic
vol_name=userdata
vol_flags=autoresize
```

Note:

- The reason why 4KB erasure is not supported is because c->leb_size < UBIFS_MIN_LEB_SZ, where UBIFS_MIN_LEB_SZ is 15KB, and leb_size is dependent on the mtd attributes.

2.12 Low-level Format Recovery Solution for UBIFS userdata Partition Issues

Repair suggestions for userdata low-level formatting:

- Store frequently written files in the same partition
- Avoid placing irreparable files in this partition
- When the partition encounters a mounting exception, repair it in the following manner:
  - Format the partition
  - Regenerate key data to support the device's continued operation

Low-level formatting command reference:

```
ubidetach -m 7
ubiformat -y /dev/mtd7
ubiattach /dev/ubi_ctrl -m 7 -d 7
ubimkvol /dev/ubi7 -N userdata -m
mount -t ubifs /dev/ubi7_0 /userdata
```

2.13 Improper Naming of UBIFS vol_name Leading to Exceptions

Exception log:

```
[    6.278922] VFS: Cannot open root device "ubi0:rootfs" or unknown-block(0,0): error -19
[    6.278943] Please append a correct "root=" boot option; here are the available
partitions:
```

Note:

- The log indicates that the rootfs partition vol_name was not defined as rootfs, leading to this issue.

2.14 Long Boot Time with UBIFS SquashFS

For SPI Nand products that require higher file system reliability and are not sensitive to boot time, it is recommended to use the static volume attribute when creating the UBI image corresponding to the squashfs file system. The power-up process can verify the integrity of the file system:

- RK's current SDK comes with this attribute by default
- If modification is needed, users should maintain data integrity checks themselves

Issue

Customers and local testing have found that the squashfs file system on the UBI image supported by spinand takes a long time to load, reaching a level of seconds per 10MB. For example, a 50MB UBI image partition takes up to 5 seconds to load.

Analysis

When creating a UBI image, you can choose between static or dynamic volume, where:

- static supports read-only and performs a crc integrity check on the entire image during the power-up mounting process, suitable for kernel/initramdisk and other images without a file system for further image verification
- dynamic supports read-write

UBIFS Official Documentation Recommendation

UBIFS Official Documentation

> Why is a dynamic volume faster to access than a static volume of the same size? Static UBI volumes were originally designed to store blobs of data like configuration files. Static volumes provide CRC-32 protection for the stored data, and static volumes know how much data they store. For example, if you have a static volume /dev/ubi0_1 of 254KiB in size consisting of 2 127KiB LEBs, and you store a 200KiB file there, and you read the whole /dev/ubi0_1, you'll get exactly 200KiB. If /dev/ubi0_1 was a dynamic volume, you'd read 254KiB. So static volumes make it easier to store blobs of data.
>
> Thus, static volumes are slower to access (compared to dynamic volumes) because UBI has to check the data CRC-32 checksum. And note, it is not a good idea to use static volumes with R/O UBIFS because UBIFS also protects all the information it stores in the UBI volume with CRC-32 checksums, and using static volumes with UBIFS would only slow down UBIFS mount speed.

The recommendation is:

- Since ubifs has its own crc check, it is not recommended to use the static volume attribute for further CRC checks in UBI images

Similarly:

- Since the squashfs file system does not perform integrity checks on the entire image during power-up, it is recommended to add the static volume attribute when creating UBI images

Test Results

TEST01: Boot Time Test:

RV1106 with spinand, using dynamic volume UBI image, supports squashfs

- Boot is normal
- No additional write data overhead during the boot process (read-only)
- Redundant CRC checks are removed, and the loading time is normal

TEST02: squashfs only has compression and decompression, without integrity checks, no static attribute added

Test method:

- Create a squashfs UBI image with dynamic volume attribute
- Locate the target file and alter some data
- Burn the image
- After booting, read the image

It was found that the file data was indeed corrupted, and ubifs did not have integrity checks.

TEST03: squashfs only has compression and decompression, without integrity checks, static attribute added, increasing check errors

Test method:

- Create a squashfs UBI image with static volume attribute
- Locate the target file and alter some data
- Burn the image
- After booting, read the image

It was found that during the power-up process, the UBI layer completed the scan and printed the corresponding error messages:

```
[    6.358100] ubi0 warning: ubi_eba_read_leb: CRC error: calculated 0xae6dc941, must be
0xf62e2df5
[    6.358212] ubi0 warning: ubi_open_volume: volume 0 on UBI device 0 is corrupted
```

Appendix: Static and Dynamic Modification Points

```
[rootfs-volume]
mode=ubi
image=rootfs.squashfs
vol_id=0
vol_size=60MiB
vol_type=static
vol_name=rootfs
vol_alignment=1
vol_flags=autoresize
```

Modified to:

```
[rootfs-volume]
mode=ubi
image=rootfs.squashfs
vol_id=0
vol_size=60MiB
vol_type=dynamic
vol_name=rootfs
vol_alignment=1
vol_flags=autoresize
```

2.15 For partitions of the same size, a UBIFS image on a 4KB-page NAND device requires more filesystem overhead than on a 2KB-page NAND device

Reasonable, as UBIFS inherently reserves a fixed number of flash blocks for algorithmic processing, hence for partitions of the same size, a 4KB-page NAND device consumes more filesystem overhead than a 2KB-page NAND device.

2.16 Insufficient UBIFS df Command Space

Refer to the UBIFS Community Description "Why does df report too little free space?" section. UBIFS needs to account for redundancy, compression, and algorithms, hence it cannot accurately estimate the partition's storable space and tends to be conservative in its estimation, which is why the df command appears to show less space, but actual transmission may be more than the df value.

2.17 Slow Mount Speed of JFFS2 File System

During the boot process of JFFS2, garbage collection is performed, which inherently results in slower operations, including the `ls` command. This performance degradation accelerates with use.

2.18 JFFS2 Partition Not Erased

Update the upgrade tool version, the new version tool will recognize the JFFS2 image and erase the partition before upgrading:

```
mount -t jffs2 /dev/block/by-name/userdata /userdata
[    2.232850] jffs2: Empty flash at 0x002c1204 ends at 0x002c2000
[    2.232931] jffs2: CLEANMARKER node found at 0x002c2000, not first node in block
(0x002c0000)
[    2.233065] jffs2: Empty flash at 0x002c200c ends at 0x002c3000
[    2.233075] jffs2: CLEANMARKER node found at 0x002c3000, not first node in block
(0x002c0000)
[    2.233428] jffs2: Empty flash at 0x002c300c ends at 0x002c4000
```

2.19 JFFS2 and SPI Nor 4K Erase Conflict

Due to JFFS2's minimum erase alignment being 8KB, and the SPI Nor kernel framework only supporting 4KB or 64KB erase alignment.

Therefore, if users need to utilize JFFS2, the SPI Nor kernel must not enable the CONFIG_MTD_SPI_NOR_USE_4K_SECTORS macro configuration, meaning only 64KB erase alignment can be used.

2.20 YAFFS File System Not Recommended

It is highly discouraged to use YAFFS, as the open-source community is no longer maintaining it. It is recommended to switch to UBIFS.

2.21 YAFFS2 File System Not Supported by RK Solution

Main Reasons:

- The RK spinand platform has robust support for UBIFS, it is recommended to use UBIFS.
- YAFFS2 involves oob-related operations, with page support for multiple data writes, currently, testing shows incompatibility.

Self-development:

- If customers require development of YAFFS, the relevant controller drivers are located in the following directories, please develop on your own:
    - For kernel 4.19 or below, drivers/rkflash/
    - For kernel 5.10, drivers/spi/spi-rockchip-sfc.c

2.22 EXT File System Nand Solution Infeasibility

Primary Reason: EXT lacks an FTL algorithm suitable for Raw-Nand, and does not support wear leveling, data refreshing, and bad block management, which are crucial algorithmic logics for Nand devices.

2.23 EXT2 File System Support for SPI Nor Flash

Emphasis:

Wear leveling is not used, use with caution, and ensure 4KB erase alignment.

Note:

- Enable the macro CONFIG_MTD_SPI_NOR_USE_4K_SECTORS.
- SPI Nor flash supports ext2; simply mount the partition on an mtdblock device.
- Avoid frequent file system write operations, as the P/E (program/erase) cycle life of SPI Nor flash is typically 100K times. Please assess the product's application lifespan accordingly.

2.24 EXT2 Lacks Discard Feature Leading to Significant Degradation in Later Read/Write Speed

The ext2 file system does not support discard, and this lack of support is also present in the latest Linux systems. Therefore, under the ext2 file system, if the flash storage is completely filled, deleting a file only modifies the inode without a discard mechanism to notify the underlying storage device to release the space accordingly. Once the storage space is fully occupied by files, it becomes impossible to release it.

Recommendations:

- Reuse files to avoid frequent creations and prevent large files from consistently occupying the entire space. If it is not possible to completely avoid this, consider regularly low-level formatting the corresponding disk; mkfs.ext2 will invoke discard to release space.
- Alternatively, migrate directly to ext4 and mount with the discard parameter.

2.25 SquashFS Compression Format and Its Efficiency

The following content serves as a preliminary reference only, as actual compression efficiency is influenced by factors such as data types:

Method	Ratio	Compression MB/s	Decompression MB/s
gzip	2.92	15	128
lzo	2.64	9.5	217
lz4	2.12	94	218
xz	3.43	5.5	35
xz 256 KB	3.53	5.4	40
zstd 1	2.71	96	210
zstd 5	2.93	69	198
zstd 10	3.01	41	225
zstd 15	3.13	11.4	224
zstd 16 256 KB	3.24	8.1	210

Data source: squashfs: Add zstd support

3. eMMC Issues

3.1 Probabilistic Communication Errors in eMMC Kernel

This issue may have various causes, and the situations encountered are listed as follows:

Insufficient External Capacitance for VDDI

The eMMC device's controller requires a working voltage of around 1.2V, which is generally obtained by stepping down from VCCQ using an LDO. The capacitor on VDDI is for filtering the 1.2V working voltage.

Most eMMC devices have a small capacitor connected internally to the VDDI pin during packaging. Different manufacturers and different models of eMMC, due to different controllers and internal capacitor values, will have varying requirements for external VDDI capacitance.

The general memory devices requirements for external VDDI capacitance are shown in the table below:

Operating Mode	VDDI Capacitance
HS400	1 - 2.2uF (A few eMMC devices may be 2-4.7uF)
Non-HS400 Mode	0.1uF - 2.2uF

When the external VDDI capacitance is too small or the capacitor fails, it may result in no issues with eMMC device A, but probabilistic communication errors when using eMMC device B.

Suggested VDDI capacitance values (upper limit or close to the upper limit):

Platform	Operating Mode	VDDI Capacitance Value
RK3399	HS400	2.2uF
Other Platforms	HS200, DDR50, SDR 50	1uF

Main Controller VCCQ Capacitance Not Connected or Too Small

eMMC 5.1 has very fast read and write speeds, and the larger the capacity, the faster the write speed. The instantaneous current of VCC and VCCQ may exceed 400mA (most eMMC devices are around 200mA). If the main controller's VCCQ capacitance is not connected or too small, it may cause the main controller's VCCQ to collapse during power-on or use, resulting in the main controller reading data errors from the eMMC end.

eMMC DATA Line Connected with ESD Devices

SD cards require ESD devices, but eMMC is soldered on the board and does not have hot-swap behavior, so it does not need ESD. Some ESD devices have large capacitance, which severely affects the signal and causes communication errors in HS400 and HS200.

eMMC CLK/CMD Routing Too Close to Other High-Speed Signals

If the eMMC CLK/CMD routing is too close to other high-speed signals (such as HDMI), communication errors may occur due to signal interference.

eMMC Routing Unequal Lengths and Serial Connection of Large Resistors on CMD or CLK

In HS200 and HS400 modes, the clock of eMMC is 200MHz. Unequal routing lengths and serial connection of large resistors on CMD or CLK can reduce the effective window for eMMC to read data. Data errors are likely to occur with voltage and temperature changes.

3.2 eMMC Firmware Upgrade Fails to Detect Storage Device or Verification Errors

The causes of this issue are varied, and the encountered situations are listed as follows:

Incorrect VCCQ Voltage Selection

VCCQ power supply supports both 1.8V and 3.3V, and the controller's IO must also be configured to match the mode, typically by selecting through an IO connected to VCC or ground. For specific details, refer to the schematic diagram.

VCCQ Voltage	Controller IO Operating Mode	Outcome
1.8V	1.8V	Normal
1.8V	3.3V	Inoperable
3.3V	1.8V	May work at low speed, but fail at high speed
3.3V	3.3V	Normal

Insufficient VDDI Capacitance

Refer to section "Insufficient External Capacitance for VDDI".

eMMC Device Soldering Issues or Damage

Some of the DNU/NC BALLs of the eMMC device need to be retained, possibly because they are internally connected to ground or VCCQ. If they are connected to other IOs during board layout, it may cause short circuits in the data lines or power supply for certain models. Soldering eMMC devices by hand may result in cold solder joint, causing the controller to not recognize the eMMC. Excessive temperature during reflow soldering or moisture damage to the eMMC device can cause device damage, making it unrecognized by the controller.

3.3 eMMC Damage or Firmware Loss After Prolonged Use

Common scenarios encountered include:

Issues with eMMC Devices

eMMC device issues generally fall into two categories:

1. A few devices have bad blocks or other reasons, leading to data corruption after a period of use (re-upgrading the controller firmware can fix this) or the eMMC device's FW errors (this requires changing the eMMC device), with the probability of such issues generally being less than one in a thousand.
2. There are issues with the eMMC's FW, causing batch data loss or FW errors, which is more likely to occur and requires FFU upgrading of the eMMC firmware or re-initialize the eMMC.

System Writes Too Much Data, Leading to eMMC Device Life Exhaustion and Damage

Application software (including Android) is designed or configured unreasonably, writing too many logs (small data random writes), which results in a significant write amplification for eMMC. For example, a log writes 1KB, but eMMC actually needs to write a page (16KB). Logs are generally written to files, and there are also updates in the file system, so the actual total write may be 3 times, 48KB.

eMMC 5.1 devices have life information in their EXT CSD, DEVICE_LIFE_TIME_EST_TYP_B [269], PRE_EOL_INFO [267], and DEVICE_LIFE_TIME_EST_TYP_A [268].

Methods for reading EXT CSD:

1. Read it yourself using the mmc-utils tool, for details refer to the help that comes with the mmc-utils tool.
2. The Linux controller reads the eMMC's ext_csd node. The location of this node may vary depending on the system; you can find it with `find ext_csd`.

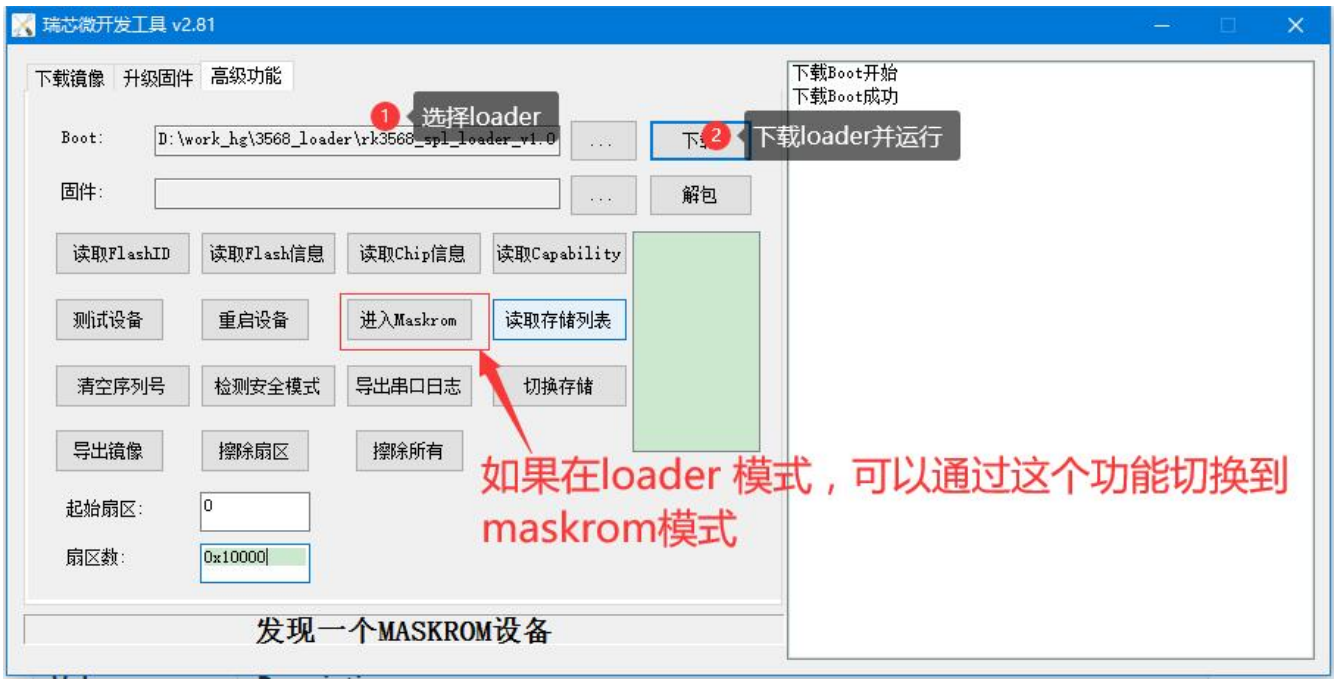
```
cat /sys/kernel/debug/mmc0/mmc0:0001/ext_csd
```

3. In maskrom upgrade mode, download the debug loader with the filename containing dump_ext_csd, and the serial port will print out the ext_csd information.

Linux upgrade tool & Android_Console_Tool

```
db xxxloaderxxx.bin
```

Android tool



EXT CSD Life Information:

DEVICE_LIFE_TIME_EST_TYP_A [268] and DEVICE_LIFE_TIME_EST_TYP_B [269] record the wear and tear life of eMMC devices, detailed information is shown in the table below:

Value	Description
0x00	Not defined
0x01	0% - 10% device lifetime used
0x02	10% - 20% device lifetime used
0x03	20% - 30% device lifetime used
0x04	30% - 40% device lifetime used
0x05	40% - 50% device lifetime used
0x06	50% - 60% device lifetime used
0x07	60% - 70% device lifetime used
0x08	70% - 80% device lifetime used
0x09	80% - 90% device lifetime used
0x0A	90% - 100% device lifetime used
0x0B	Exceeded its maximum estimated device life time
Others	Reserved

PRE_EOL_INFO [267] records whether the reserved blocks of the eMMC device are exhausted, detailed information is shown in the table below:

Value	Pre-EOL Info.	Description
0x00	Not Defined	
0x01	Normal	Normal
0x02	Warning	Consumed 80% of reserved block
0x03	Urgent	
0x04 ~ 0xFF	Reserved	

Abnormal Power Loss During OTA Upgrade Process

After ANDROID 7.1, OTA firmware upgrades are directly block updates for the system partition (not file-based updates as before). If there is an abnormal power loss during the OTA upgrade process, it may cause data corruption in the system partition.

3.4 eMMC Firmware Upgrade Stuck in Maskrom Mode

eMMC Not Properly Soldered or PCB Issues

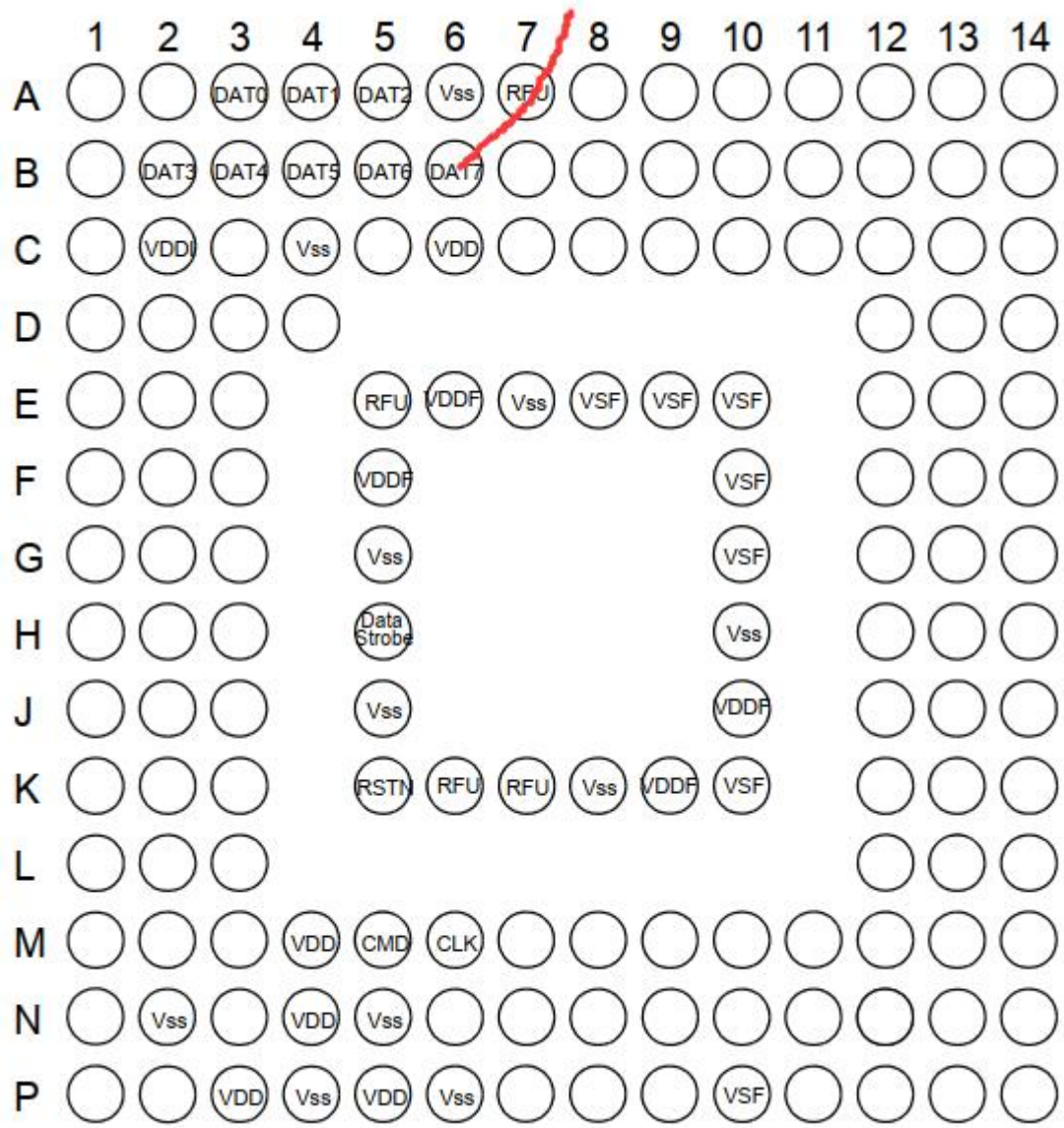
BOOTROM initiates with an 8-line mode; if the eMMC D1-D7 lines are not properly connected, the 8-line mode cannot be activated.

Poor Soldering:

Some individual samples have issues, usually caused by poor soldering.

PCB Issues:

All samples have problems, generally due to PCB issues, such as routing data lines through the RFU or DNU balls.



Unprogrammed uboot or trust:

If uboot or trust is not programmed, or if the programming location is incorrect, the miniloader cannot boot into uboot and will return to the maskrom upgrade mode. Serial print information will show that it has booted into the miniloader and then restarted.

4. Flash Issues

4.1 Flash Read/Write Speed

General Testing (Close to Optimal):

The dd tool tests at least 10MB of space.

Flash Type	Test Environment	Sequential Read	Sequential Write
SLC Nand	Linux	16MB/s (IO 30MHz)	4MB/s (IO 30MHz)
SPI Nand	Linux	10.8MB/s (IO 80MHz)	4.0MB/s (IO 48MHz)
SPI Nor	Linux	24.0MB/s (IO 80MHz)	100~200KB/s (IO 48MHz)
SPI Nor	RTT	35MB/s (IO 80MHz)	100~200KB/s (IO 80MHz)

Flash Type	Test Environment	Random 4K Read	Random 4K Write
SPI Nor	RKOS	13MB/s (IO 80MHz)	100~200KB/s (IO 48MHz)

Special Test Environment Write Speed 1 (Close to Saturation)

Test Environment: RK3308_V10_EVB

Kernel: 4.4 64-bit

User Partition: 352MB

FLASH Size: 512MB

Test case:

```
import serial
import time

ser = serial.Serial('/dev/ttyUSB0', 115200)
while True:
    ser.write(b'rm /userdata/test && sync\r')
    time.sleep(2)
    ser.write(b'time dd if=/dev/zero of=/userdata/test bs=1M count=350 && sync\r')
    time.sleep(30)
```

Test Conclusion: After testing for half an hour, the full load write speed is 2.0 MB/s.

4.2 Flash Write Protection Feature

Flash Write Protection Feature

- Software write protection, Flash devices typically support but implementation varies.
- Hardware write protection via the WP# pin is supported only by certain memory devices. The RK SoC cannot directly control this pin. If this feature is required, the WP# pin must be routed to and controlled by a GPIO.

RK Storage Solution Does Not Add Write Protection Support

Since the read and write of memory are unified interfaces, theoretically, there will be no abnormal signals that can trigger complete read and write operations, and the protection schemes of different memory devices are vastly different, so no corresponding write protection processing is added.

4.3 Incompatibility of Two Flash Devices Selected by Software

For stacked-die flash devices consisting of two identical flash dies, software-controlled device select is not supported by the current software framework.

4.4 Common Prints for Failed Flash Kernel Storage Initialization

Assuming that the storage initialization is abnormal (abnormalities would definitely not correspond to the creation of mtd partitions), the system will continue to run until it stops when mounting the root file system that is not a ramdisk, and reports prints related to waiting for root...

4.5 Ensuring Data Persistence in Flash Storage Device Nodes

When using `open()`, add the `O_SYNC` or `O_DIRECT` flag, or after writing data, add `fsync`. If upgrading the image using the `dd` command, it is recommended to include the following flags:

- `iflag=nocache` and `oflag=direct`

4.6 Flash Material Replacement Instructions

Differences between TA/SA Materials in the Support List

The RK SPI Flash controller IP is a compatibility IP, where T/A refers to samples that have been validated by the storage verification platform, and S/A refers to the compatibility platform. Both T/A and S/A flash materials can be imported into the product.

Regarding New Nand Flash Materials

RK's flash device verification mainly focuses on basic functionality and the stability of the sent samples on the test platform. Therefore, it is recommended to conduct further testing on the flash for specific product devices, including:

- Signal Consistency: There should be no significant difference in signals before and after replacing the flash, and it should meet the spec requirements of the device.
- Abnormal Power Loss Robustness: Refer to the document **Rockchip_Developer_Guide_Linux_Flash_Open_Source_Solution_EN**

4.7 Troubleshooting Flash TFTP Firmware Download and Burning Anomalies

Initial problem localization:

- Add md5sum information at the following stages and compare:
    - Add md5sum after creating the upgrade image

    - TFTP firmware download, firmware adds md5sum
        - If the comparison is abnormal, suspect network or DRAM issues
    - Flash firmware burning and online read-back, add md5sum to the read-back image
        - If the comparison is abnormal, suspect Flash operation anomalies, recommend reducing Flash speed and switching to single-line transmission for further confirmation
    - Device enters maskrom mode, add md5sum to the read-back image
        - Analyze in conjunction with "Flash firmware burning and online read-back"

If Flash issues are initially suspected:

- Effective speed reduction and single-line transmission, recommend measuring signal quality, measuring whether Flash power supply is unstable, adjust hardware to see if default configuration can work
- If speed reduction and single-line transmission are ineffective, contact RK's storage software engineer for further localization

4.8 Flash uboot Phase MTD Interface Comparison with MTD_BLOCK Interface



P/E Granularity Mismatch Issue:

Like NAND Flash, NOR Flash also has a discrepancy between the minimum erase (Erase) and minimum write granularity (Program), so it is necessary to consider the combination of P/E. For example: writing 256B into a 4K sector of NOR requires erasing the 4K first, and the data outside of the 256B address (4KB - 256B) must be saved in advance and rewritten.

MTD and Block Interfaces:

- The MTD interface requires the upper layer to manage P/E on its own.
- The block interface abstracts the logic of the minimum read/write logical sector, and reading/writing this sector will not affect other sectors (internally handling P/E granularity mismatch).
    - The Linux framework provides an mtdblock interface, mapping the logical sector to 512B (it can be imagined that the interface will maintain the P/E relationship, avoiding P/E granularity mismatch issues when writing into a 512B sector).
    - RK SDK u-boot completes the corresponding logic by referencing the Linux framework.

4.9 Flash Cold Temperature Test Anomaly

It is recommended to first measure whether the VCCIO Flash sel pull-up and pull-down voltage levels are abnormal at low temperatures.

4.10 Flash Development Board Maskrom Button Proximity to Touch Affects Data Transmission

The maskrom button should be designed in a position that is not easily touched to prevent accidental contact from affecting data transmission.

5. Flash Controller Issues

5.1 Controller FSPI Clock Duty Cycle Below 50%

The output clock is derived from a specific PLL in the clock tree. If the division is an odd number, the clock duty cycle may not be 50%, leading to critical timing for peripheral devices at the highest frequency. An even number division results in a 50% duty cycle.

5.2 Controller FSPI Non-General SPI Controller, Only Supports SPI Memory Protocol

FSPI (formerly known as SFC) only supports applications that comply with the SPI memory timing, which is cmd + addr (optional) + dummy (optional) + data (optional). In particular, "it does not support operations that directly input after cs assert". Please carefully check the timing sequence.

If you need to extend peripherals such as FPGA, please assess whether they meet the extension requirements yourself. For kernel driver reference, see spi-rockchip-sfc.c.

5.3 Controller FSPI Establishment and Hold Time Not Measured

uboot and kernel stages FSPI read data are configured for four-line transmission by default, and if the rate is configured to be consistent, the Host read sampling establishment and hold time requirements should theoretically be the same.

Background

1. FSPI Read Sampling Basics

FSPI operates in SPI mode 0 (CPOL clock is low during idle, CPHA samples on the first edge), and based on this, a half-cycle read sampling is pushed ——— sampling on the negative edge.

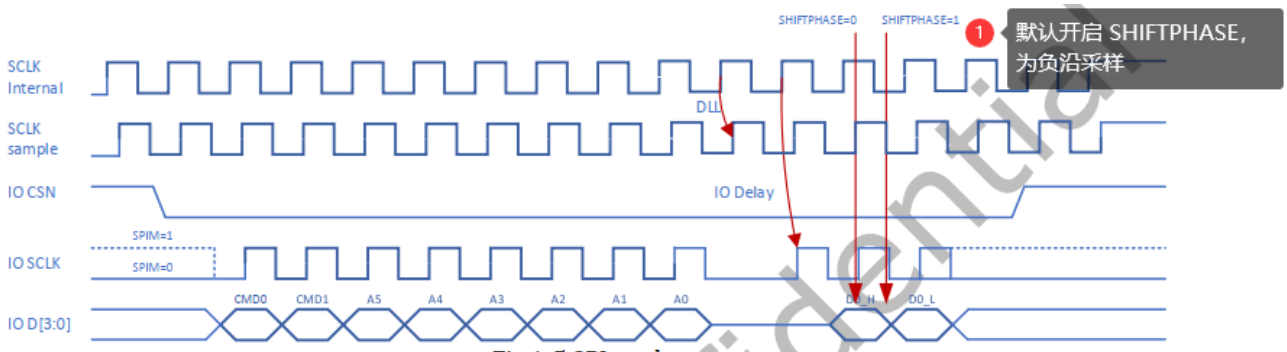


Fig.1-5 SPI mode

fidential

2. DLL (Delay Line) Requirements

In the case of IO timing delay (FSPI timing constraints) and the combined output delay of most devices, the read data signal's establishment and hold time are critical, so it is necessary to add a delay line to support obtaining the best establishment and hold time.

3. DLL Features

The RK host controller FSPI supports read sampling with a delay line:

- DLL is superimposed internally on the chip, so the IO cannot measure this value
- DLL is affected by PVT and process, so it is not a fixed value, nor is it phase-locked
- The delay effect is stacked on the negative edge sampling in "FSPI Read Sampling Basics"

In summary, the specific delay of DLL cannot be calculated and will not be reflected in the signal.

4. DLL Software Strategy

When the IO rate exceeds 50MHz, it is enabled, including the following features:

- Tuning is performed in all software stages
- Comparing flash ID values

Summary

The main reason for not measuring the establishment and hold time of RK Host FSPI read data is:

- FSPI IO working clock is mostly above 50MHz, enabling DLL, so the actual read data establishment and hold time cannot be measured, refer to the "DLL Features" description
- Below 50MHz, based on "FSPI Read Sampling Basics," there is a large margin of redundancy

5.4 Does the FSPI controller support connecting different devices to separate chip-select lines?

If the FSPI controller in this SoC is configured to extend the CS1N signal, the FSPI IP theoretically supports dual chip-select operation. However, only the device connected to CS0N can serve as the bootdev, in addition, devices connected to the two chip-select lines share a significant number of IO signals, which may introduce signal interference and affect the operating frequency of the devices. Therefore, a comprehensive hardware and software evaluation is recommended before adopting this configuration.

6. NAND Issues

6.1 PX30 Nand Device Modified to rkflash Storage Driver

For an introduction to the rk_nand and rk_flash storage frameworks, please refer to the "Rockchip_Developer_Guide_UBoot_Nextdev_CN" document, specifically the Storage chapter. The rk_nand is primarily used for MLC, TLC, and other high-capacity Nand flash products, while rk_flash is applied to SLC Nand flash products. Since the SDK provided by PX30 mostly includes firmware for high-capacity Nand, switching to SLC Nand flash requires corresponding changes to the storage driver.

Main modification points:

1. rkbin should use the RKBOOT/PX30MINIALL_SLC.ini configuration to package the loader firmware.
2. u-boot should refer to the "Rockchip_Developer_Guide_UBoot_Nextdev_CN" Storage chapter, and change the RKNAND driver to RKFLASH.
3. The kernel should be modified similarly to u-boot, changing RK_NAND to RK_FLASH, and enabling the nandc0 node in the dts.

Possible modification exceptions:

Q1: Can u-boot not boot normally?

A1: It may be due to the storage driver or dts not being configured successfully. Enter the following commands for confirmation:

```
=> dm tree
...
# The normal device should have a similar printout
rksfc [ + ] rksfc |-- sfc@ff4c0000 **
blk [ + ] rkflash_blk | |-- sfc@ff4c0000.spinand.blk
blk [ ] rkflash_blk | `-- sfc@ff4c0000.spinor.blk
```

6.2 Nand uboot Custom Read/Write Behavior Standards

If the number of erase cycles is not frequent, you can consider directly calling the mtd interface for erase and read operations. However, some customers have the need to frequently record information under uboot, with the following scenarios:

- Bare partitions without a file system
- Frequent logging
- Do not wish to integrate into the vendor partition to avoid impact

To reduce the impact of in-place erasing on the lifespan of Nand products, it is recommended to:

- Add a special partition, refer to the vendor partition for read and write implementation, source code reference: arch/arm/mach-rockchip/vendor.c, interface reference mtd_vendor_storage_init and mtd_vendor_write.

Introduction to the principle of vendor partition implementation:

- The initialization process detects whether there is a vnvm partition, and if it exists, register the vendor interface
- Write two copies at a time, each write unit is aligned to the page size, and the write position is the first page of the block, erasing the entire block.

6.3 NAND SFTL Space Planning

Device	idblock	user data	reserve data	sys block	redundancy
128MB	2MB	107MB	11MB (14.8%)	8MB	8.6%
256MB	2MB	218MB	24MB (14%)	12MB	4.7%
512MB	2MB	464MB	31MB (9%)	15MB	2.9%

Where 11M is used for flash storage exchange, enhancing read-write efficiency, and sys block is dedicated to FTL.

6.4 NAND Flash Pre-Burning Does Not Allow Combining Separate Images for One-Time Burning

In principle, the final implementation of the burner must be separate burning:

- The flash software's bad block management during operation is limited to the corresponding partition, so the pre-burning behavior should be consistent with it.

Some burners support special packaging methods developed in-house or from external open-source:

- Independent images are packaged into an all_in_one data package (which may come with partition information), and then the burner parses the unique partition information, unpacks, and ultimately performs separate burning.
    - Please provide a demo from the burner manufacturer or develop it yourself.

RK has analyzed the all-in-one packaging method of a burner:

- For detailed reference, see the spinand burner production document.

  20210804_flash_programmer

6.5 NAND Open Source Framework Bad Block Table Management

The Need for Bad Block Table Management

Almost all operations of Nand flash need to be aware of whether the flash block to be operated is a bad block. If the bad block information is not recorded, it will be necessary to re-evaluate whether there is a bad block mark in the bad block mark area of the flash page each time, which will lead to the following problems:

- Scanning the flash bad block mark area is an IO read operation, which requires a certain amount of time;
- The reliability of recording bad block marks on bad blocks that appear during use is limited, and flash blocks may not be able to successfully mark recognizable bad block marks under unstable conditions.

The mtd framework will cache the bad block information obtained for the first time from IO, and then the bad block information of the flash block at the same location can be directly obtained from the bad block table cache when reading again. However, after a restart and power-up, the bad block table cache has to be re-established, so in addition to this, the bad block table should also be stored in non-volatile storage. (bbt cache reference drivers/mtd/nand/bbt.c)

SLC Nand bbt in flash

In the open source framework of SLC Nand, there is an integrated method for managing bad block tables that can be written back to flash, centralizing and updating bad block information.

SPI Nand bbt in flash

The open source framework does not support the bad block management method that can be written back to flash for SPI Nand, therefore, rk provides corresponding bad block management support, which is enabled by default, detailed code:

u-boot:

```
drivers/mtd/nand/bbt.c
```

kernel:

```
drivers/rkflash/sfc_nand_mtd_bbt.c
```

This bad block table management method has the following characteristics:

1. It combines with the existing bbt.cache framework part of drivers/mtd/nand/bbt.c in the open source framework that caches flash bad block information;
2. Nand flash should reserve the last 4 blocks for bad block table storage, and the last partition should be adjusted accordingly;
3. Generate the bad block table upon the first power-up in the u-boot environment;
4. Write back two copies of the bad block table each time;
5. In u-boot, bad blocks do not involve deadlock issues because there are no real mutexes and multi-threading situations.

7. PP SLC NAND Issues

7.1 PX30 SLC Nand IDB Creation Issues

Different image creation methods vary:

1. IDB Image Creation: The burning tool exports the idb image with oob, located at flash block 0, typically with a length of 2 flash blocks.
2. Other Image Creation: Utilize programmer_image_tool with the -l parameter to modify the chain list link method.

Note:

- The px30 IDB burning tool image is created separately, mainly because the old platform added 1 page of flash info information to the slc nand IDB image area and included special ECC, which is not integrated into programmer_image_tool (the new platform does not have this flash info), hence it is incompatible.

7.2 Analysis of SLC NAND devices from Original Manufacturers Reveals Continuous Large Numbers of Bad Blocks

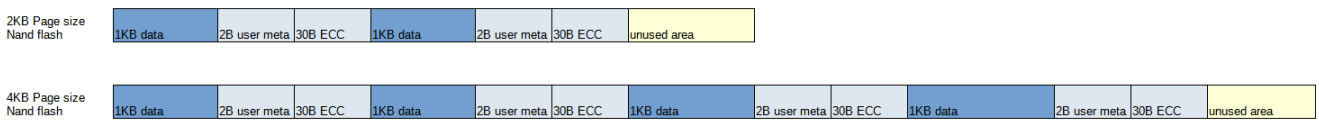
Customers or original manufacturers may consider blocks marked as bad blocks after disassembling SLC NAND flash and inspecting them with programmers or other tools.

In fact, this judgment method is incorrect. For SLC NAND used by RK FTL, RK AP manages bad blocks with a bad block table, and blocks that have been used cannot be judged again based on the value of spare 0 as a bad block marker. Therefore, using third-party tools to judge after disassembling the device is inaccurate, and actual issues encountered in use should be analyzed based on specific circumstances.

7.3 SLC NAND Flash Programmer Does Not Support Master Device Images

Assuming "image" refers only to data:

1. SPI NAND relies on the device-side ECC, so the image that users flash is only data, with ECC depending on the device-side.
2. SLC NAND relies on the controller-side generated BCH code for ECC protection, and one page after RK BCH processing, the data and oob (including metadata and BCH code) will be stored according to the following pattern:



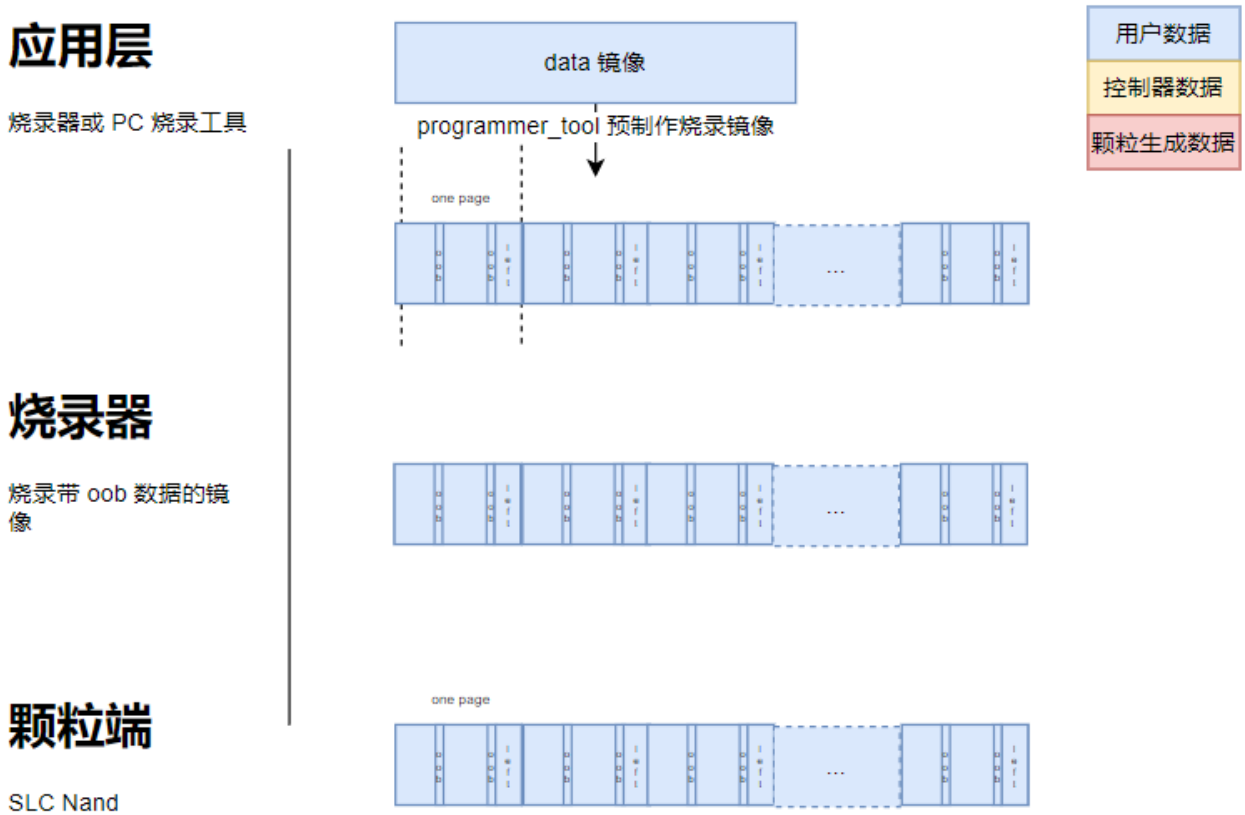
Note:

- Re-layout: The original default layout of data + oob will be reorganized into a scattered layout of data + oob + data + oob..., with the original oob first byte bad block marker repositioned within the data area:
    - After flashing the firmware, the SLC NAND cannot re-identify bad blocks according to the factory's default bad block marking method, which is the first oob byte.
    - The RK software identifies bad blocks by first confirming BCH failure, then confirming that the first oob byte is a bad block marker.

This ultimately leads to the programmer being unable to normally skip bad blocks when reading data back.

7.4 SLC NAND Programmer Burning Images with Valid Data and Redundant OOB Data

OOB data (meta data and BCH) is added at this layer when creating the image, and the burning of the image is set to "burn image as data with OOB".

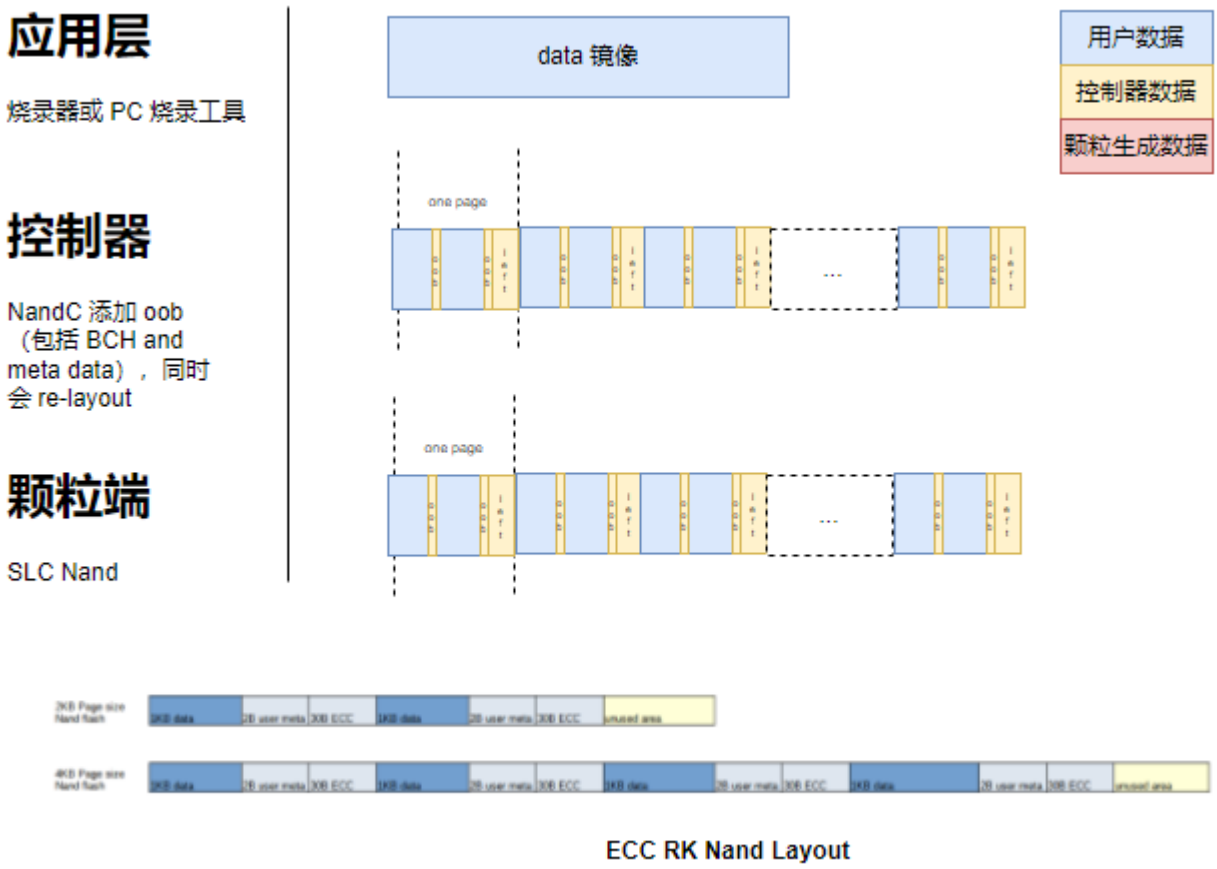




ECC RK Nand Layout

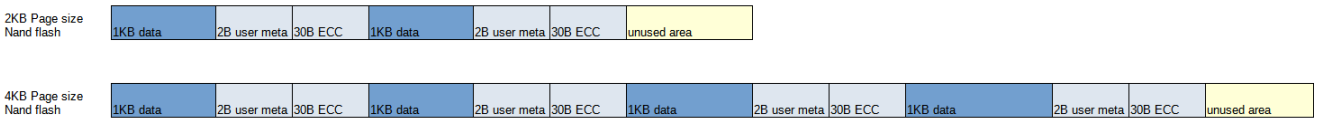
7.5 SLC NAND SDK Outputs Valid Data Image (Excluding Redundant OOB Data)

OOB data (meta data and BCH) is added at the controller level, and the image used for user flashing contains only data.



7.6 SLC NAND Software ECC Layout

The RK SDK slc NAND (parallel) utilizes the BCH module integrated within the host controller for ECC, configured as 16bits/1KB. The layout is depicted in the following figure:



7.7 SLC NAND rkflash Solution Kernel Phase Read and Write Speed Optimization

Sacrificing CPU Scheduling:

1. Update the storage patch, link:

   Storage Patch

2. Modify the `usleep_range` in the `drivers/rkflash/nandc.c` driver from 20-30 to 5-10 —— Adjusting the `usleep` granularity to optimize CPU thread scheduling (but this will sacrifice some transfer rate). Conversely, increasing the `usleep` duration will reduce the CPU load under high IO throughput conditions.

7.8 SLC NAND mtd Solution SPL Phase Read Transfer Rate Optimization

Background

Due to the non-uniform standards of PP NAND flash devices, the configuration of the NandC controller tends to be more conservative, pursuing stability. If the customer's product has higher requirements for speed, appropriate acceleration optimization can be performed.

Patch Update Confirmation

Taking RV1126 as an example, patches are added to uboot, and the SPL is compiled and updated to support a 30MHz IO rate:

```
diff --git a/arch/arm/mach-rockchip/rv1126/rv1126.c b/arch/arm/mach-rockchip/rv1126/rv1126.c
index 84df82ab138..e3ae330e413 100644
--- a/arch/arm/mach-rockchip/rv1126/rv1126.c
+++ b/arch/arm/mach-rockchip/rv1126/rv1126.c
@@ -77,6 +77,7 @@ DECLARE_GLOBAL_DATA_PTR;
 #define CRU_CLKSEL_CON50       0x1c8
 #define CRU_CLKSEL_CON51       0x1cc
 #define CRU_CLKSEL_CON54       0x1d8
+#define CRU_CLKSEL_CON59       0x1ec
 #define CRU_CLKSEL_CON61       0x1f4
 #define CRU_CLKSEL_CON63       0x1fc
 #define CRU_CLKSEL_CON65       0x204
@@ -749,6 +750,11 @@ int arch_cpu_init(void)
        udelay(1);
        writel(0x00100000, CRU_BASE + CRU_SOFTRST_CON10);

+#ifdef CONFIG_SPL_BUILD
+       writel(0x000ff004, CRU_BASE + CRU_CLKSEL_CON59);
+       printf("NandC gpll=768,div=(%d+1)\n", readl(CRU_BASE + CRU_CLKSEL_CON59) & 0xff);
+#endif
+
        return 0;
 }
 #endif
diff --git a/drivers/mtd/nand/raw/rockchip_nand_spl.c
b/drivers/mtd/nand/raw/rockchip_nand_spl.c
index 6e39287ef39..d6ed7038914 100644
--- a/drivers/mtd/nand/raw/rockchip_nand_spl.c
+++ b/drivers/mtd/nand/raw/rockchip_nand_spl.c
@@ -92,7 +92,7 @@ static u32 nand_block_num;

 static void nandc_init(struct rk_nand *rknand)
 {
-       writel(0x1081, rknand->regs + NANDC_REG_V6_FMWAIT);
+       writel(0x1061, rknand->regs + NANDC_REG_V6_FMWAIT);
 }

 static void rockchip_nand_wait_dev_ready(void __iomem *regs)
```

Requirements

Due to the acceleration of NandC during the SPL phase, it is recommended to confirm the following:

1. Signal quality, confirm whether the output signal meets the device spec requirements
2. Burn-in stress test, direct machine reboot test can be performed

7.9 SLC NAND Excessive Bad Blocks at the End MTD Solution Device Failure to Operate Normally

If the kernel version is 4.19, please first confirm whether the following patch has been updated:

```
commit e323080273b1e6ad2862158c2c634bf9f36671b2
Author: Jon Lin <jon.lin@rockchip.com>
Date:   Tue Aug 29 15:56:56 2023 +0800

    mtd: rawnand: rockchip: Modify the judgment of the last page

    Incorrect judgment without affecting bad block judgment logic.

    Change-Id: I123be12cc50e52e6a24c4276a5ac68208239af42
    Signed-off-by: Jon Lin <jon.lin@rock-chips.com>
```

8. SPI Nand Common Issues

8.1 SPI Nand Flash Devices Plane Select Material Compatibility Issues

Issue:

The RV1126/RV1109 SoCs have compatibility issues with some 2-plane physical structure SPI Nand particles, requiring the trimming of IDB size to be brought up, hence it is not recommended to use such particles, which will be reflected on the support list.

Compatibility Solution:

If it is necessary to use such particles, RK has made special compatibility processing, that is, trimming the SPL for corresponding support, and the trimmed defconfig is configs/rv1126-spl-spi-nand_defconfig.

Main trimmings:

1. Support for other storage types: parallel NAND, eMMC, SPI Nor, SD card
2. GPT support
3. AB partitioning

If there are requirements for the above functionalities, consider further trimming to enable these features, removing the following unused original particle definitions:

```
CONFIG_MTD_SPI_NAND=y
CONFIG_SPI_NAND_GIGADEVICE=y
CONFIG_SPI_NAND_MACRONIX=y
CONFIG_SPI_NAND_MICRON=y
CONFIG_SPI_NAND_TOSHIBA=y
CONFIG_SPI_NAND_WINBOND=y
CONFIG_SPI_NAND_DOSILICON=y
CONFIG_SPI_NAND_ESMT=y
CONFIG_SPI_NAND_XTX=y
CONFIG_SPI_NAND_HYF=y
CONFIG_SPI_NAND_FMSH=y
CONFIG_SPI_NAND_FORESEE=y
CONFIG_SPI_NAND_BIWIN=y
```

Confirm the Combined Size of spl.bin + ddr.bin Meets the Requirements:

In the uboot directory, enter the following command, replacing ddr.bin with the target

```
./tools/mkimage -n rv1126 -T rksd -d ./../rkbin/bin/rv11/rv1126_ddr_924MHz_v1.05.bin:./spl/u-boot-spl.bin rv1126_idb1.bin
```

Known Particles with This Issue:

Particle Silk Screen	ID0	ID1
MT29F2G01ABA, XT26G02E, F50L2G41XA	2C	24
DS35Q2GA-IB	E5	72
DS35M2GA-IB	E5	22
FM25S02A	A1	E5
DS35Q2GB-IB	E5	F2
DS35Q4GM	E5	F4

Further Explanation:

The bootrom cannot scan the data on the odd-numbered blocks 1, 3, 5... of these particles.

A block is 128KB

So once the IDB size exceeds 128KB, it will inevitably span blocks, and the bootrom will definitely not be able to scan the IDB part of the data, and cannot load the IDB correctly.

8.2 1V8 SPI Nand Flash Devices Selection

1V8 Flash devices are not difficult to find, as memory original manufacturers typically have 1V8 products. RK already supports 7 memory original manufacturers, based on the latest flash support list.

1V8 Flash devices supporting, in addition to domain configuration and IO power supply needing to be properly converted, are compatible with 3V3 devices in storage drivers, with no special configurations required:

Date	Vendor	Model	Density (Gbit)	Capacity (MB)	Voltage (V)	Type
20230705	XTX	XT26Q04DWSIGA	4GBits	512MB	1.7V-1.95V	SPI NAND
20230705	XTX	XT26Q01DWSIGA	1GBits	128MB	1.7V-1.95V	SPI NAND
20230705	XTX	XT26Q02DWSIGA	2GBits	256MB	1.7V-1.95V	SPI NAND
20230705	FORESEE	F35UQA002G-WWT	2GBits	256MB	1.7V-1.95V	SPI NAND
20230705	FORESEE	F35UQA001G-WWT	1GBits	128MB	1.7V-1.95V	SPI NAND
20230317	HYF	HYF1GQ4IDACAE	1GBits	128MB	1.70V-1.98V	SPI NAND
20230317	HYF	HYF2GQ4IAACAE	2GBits	256MB	1.70V-1.98V	SPI NAND
20230217	Dosilicon	DS35M12B-IB	512MBits	64MB	1.7V-1.95V	SPI NAND
20220901	MXIC	MX35UF1GE4AD-Z4I	1GBits	128MB	1.7V-1.95V	SPI NAND
20220818	MXIC	MX35UF4GE4AD-Z4I	4GBits	512MB	1.7V-1.95V	SPI NAND
20220818	MXIC	MX35UF2GE4AD-Z4I	2GBits	256MB	1.7V-1.95V	SPI NAND
20210918	GigaDevice	GD5F2GM7REYIGY	2GBits	256MB	1.7V-2.0V	SPI NAND
20210911	Dosilicon	DS35M2GA-IBR	2GBits	256MB	1.7V-2.0V	SPI NAND
20210911	Dosilicon	DS35M1GA-IBR	1GBits	128MB	1.7V-1.95V	SPI NAND
20210911	Dosilicon	DS35M1GB-IB	1GBits	128MB	1.7V-1.95V	SPI NAND
20210911	GigaDevice	GD5F2GQ5REYIG	2GBits	256MB	1.7V-2.0V	SPI NAND
20210911	GigaDevice	GD5F1GQ5REYIG	1GBits	128MB	1.7V-2.0V	SPI NAND
20200701	Dosilicon	DS35M1GA-IBR	1GBits	128MB	1.65V-1.95V	SPI NAND
20200831	MXIC	MX35UF1GE4AC-Z4I	1GBits	128MB	1.7V-1.95V	SPI NAND
20200831	MXIC	MX35UF2GE4AC-Z4I	2GBits	256MB	1.7V-1.95V	SPI NAND

8.3 SPI Nand Flash HYF1GQ4UTXCAE Anomaly Power Down Issue

The original Manufacturer's HYF1GQ4UTXCAE device experienced an abnormal power down failure, confirmed to be a series of devices upstream lacking adjustment for HWP_EN support.

```
commit 72986913fc4ba3e0c1c276aa63e2040ff8290201
Author: Jon Lin <jon.lin@rock-chips.com>
Date:   Mon Aug 14 14:52:36 2023 +0800

    mtd: spinand: Enable HWP_EN for skyhigh devices

    HWP_EN must be enabled first before block unlock region is set.

    Change-Id: I6b107d97de48bb2644da865f353d2adace95224e
    Signed-off-by: Jon Lin <jon.lin@rock-chips.com>
```

8.4 Preliminary Research on SPI Nand Software Soft ECC Solution

It should be clarified at the outset that software ECC offers almost no assistance in error correction and is somewhat helpful in determining the validity of data, but the RK solution only utilizes the hardware ECC provided by the memory device itself.

Introduction to ECC:

- Data from spinand requires ECC verification because there is a certain probability of bit flips occurring in the Nand medium.
- It is typically hardware ECC, possibly integrated into the memory device (most devices have integrated ECC), or integrated into the host (such as HiSilicon devices).

Requirements for Software ECC:

If SPI Nand does not have integrated hardware ECC, the RK solution defaults to not supporting such memory devices because their reliability is extremely low. If the memory device already has hardware ECC, and software ECC is still being considered:

- It offers almost no assistance in error correction. If the memory device's hardware ECC fails in error correction, it usually indicates a large-scale anomaly, and the minor error correction by software would be of no help.
- It can serve as a flag for determining whether the data is valid because, in extreme cases, anomalies may occur in the transmission path (such as abnormal power loss). In such cases, hardware ECC may not report an error, so a data verification mechanism similar to that within UBIFS is necessary.

Application Scenarios for Software ECC:

For partitions without a file system, such as misc, if SPI Nand read/write operations are performed, it is possible to consider adding an ECC mechanism for data protection in extreme conditions, or to adopt other effective data protection mechanisms.

8.5 SPI Nand Software BBT Store in Flash Protection Mechanism

Background

The RK SPI Nand SDK supports storing flash bad block information in the BBT table, for details refer to the "Bad Block Table Management Requirements" section of this document.

Early Solution

The early verification of the bad block table mainly relied on:

- the first byte of the out-of-band (oob) data
- the magic number stored at the end of the valid data in the BBT

Corresponding patch versions:

u-boot:

```
commit 78cac1dffdc0bd64e47294e41d2432b7760e7186
Author: Jon Lin <jon.lin@rock-chips.com>
Date:   Sun Jun 28 20:13:39 2020 +0800

    mtd: nand: Remove bbt option property if scan fail

    Change-Id: Ifb5b500b6ffee551aea5b6aecea629b3d0ea6207
    Signed-off-by: Jon Lin <jon.lin@rock-chips.com>
```

kernel:

```
commit fe246b156492ee509bb1d071c176d324542d3193
Author: Jon Lin <jon.lin@rock-chips.com>
Date:   Thu Dec 17 16:05:27 2020 +0800

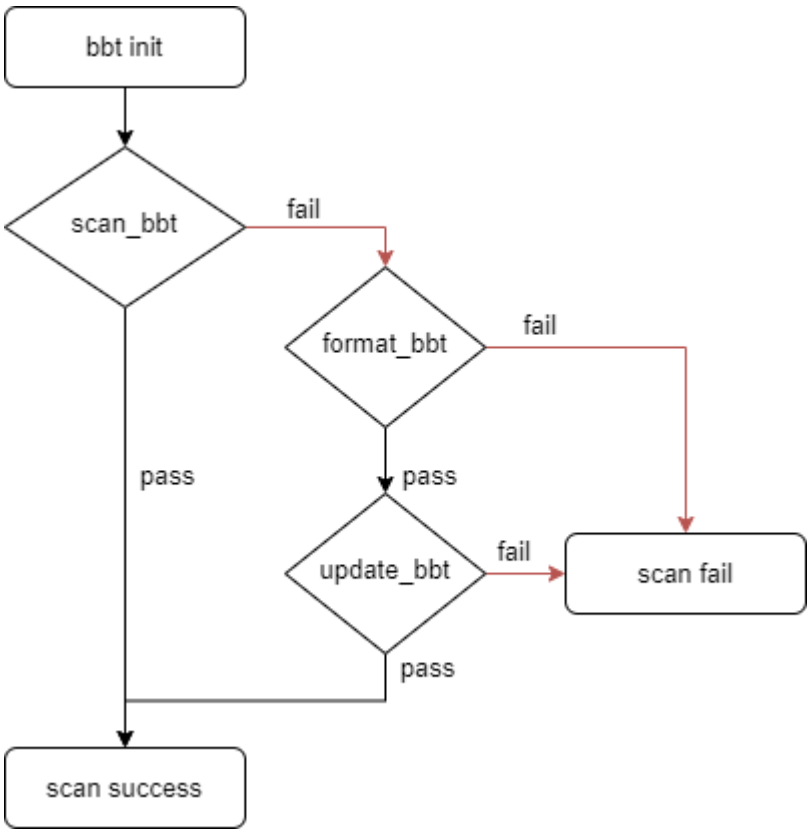
    drivers: rkflash: Fixed bbt operation calculation error

    Fixes: 008340a82 ("drivers: rkflash: Change to use the api which the oob area available")
    Change-Id: I140aa76a2acb73271ba04b7060030dc06a2353e6
    Signed-off-by: Jon Lin <jon.lin@rock-chips.com>
```

Optimization Solution

Considering the instability of SPI Nand, although the root cause is not from the bad block table logic itself, the bad block table may be subject to destructive impacts, hence targeted boundary problem protection mechanisms are implemented.

For the BBT framework:



The following protections are added:

- A hash value is added to verify the bad block table information
- When `format_bbt` scans the flash for bad blocks, a secondary verification mechanism is added for suspicious blocks identified as bad blocks
- When `update_bbt` updates the bad block table, a read-back verification mechanism is added when writing the bad block table, and redundant information is erased if the verification fails

Optimization Solution Error Injection

Inject the following errors during the uboot/kernel phase:

- Simulate exceptions when scanning the bad block table during power-on
- Simulate exceptions when scanning the flash for bad blocks during the BBT low-level formatting process
- Simulate write exceptions, hash value exceptions, read-back exceptions, and read-back hash value exceptions during the BBT write process

Other Notes:

- Since the bad block table is not supported during the SPL phase, there are no destructive actions, hence no error injection tests are performed
- After stress testing, the bad block table remains normal with no erroneous bad block information

Optimization Solution Patches

uboot:

```
commit 55ef8700b677919efa06f3bfe1dcab3a4554a92d
Author: Jon Lin <jon.lin@rock-chips.com>
Date:   Sun Sep 24 20:11:20 2023 +0800

    mtd: nand: bbt: Add mechanisms to protect bad block tables

    1.Add hash check
    2.Add anti-shake mechanism to avoid damaging the bad block tables
    3.Add anti-shake mechanism to optimize the reliability of bad block table

    Change-Id: I6ac9554ae0f642d6a89d21a8f79a2a87d607f993
    Signed-off-by: Jon Lin <jon.lin@rock-chips.com>
```

kernel:

```
commit cc1f433fb9ade372b06e9d62651ed8d719aaac94
Author: Jon Lin <jon.lin@rock-chips.com>
Date:   Sun Sep 24 20:24:00 2023 +0800

    drivers: rkflash: Add mechanisms to protect bad block tables

    1.Add hash check
    2.Add anti-shake mechanism to avoid damaging the bad block tables
    3.Add anti-shake mechanism to optimize the reliability of bad block table

    Change-Id: I888ceba54e5bbc55283850316e27560c484a9cf5
    Signed-off-by: Jon Lin <jon.lin@rock-chips.com>
```

8.6 The SPI NAND programmer supports programming the master device, but does not support exporting the entire image in a single operation

It is recommended to use the RK Programmer Image Tool to generate programmer images instead of creating them from a master device.

Exporting the entire SPI NAND image as a master device image in a single operation is not supported because bad block management in the Linux MTD framework, UBIFS, and related software components is performed on a per-partition basis.

Example: Partition definition: 8MB a partition (with a bad block of 128KB in the middle) | 8MB b partition

Then do not burn: 7MB A image and 7MB B image,

Assuming a total image of 7MB_A + 1MB redundancy + 7MB_B, when burned, after skipping the bad block:

The starting address of A image's valid part is 0

The starting address of B image's valid part is 8MB + 128KB

However, the software-defined partition starts at 8MB, so it will begin loading data from the starting address of the 8MB flash offset, causing the image to be misaligned.

Correct Approach:

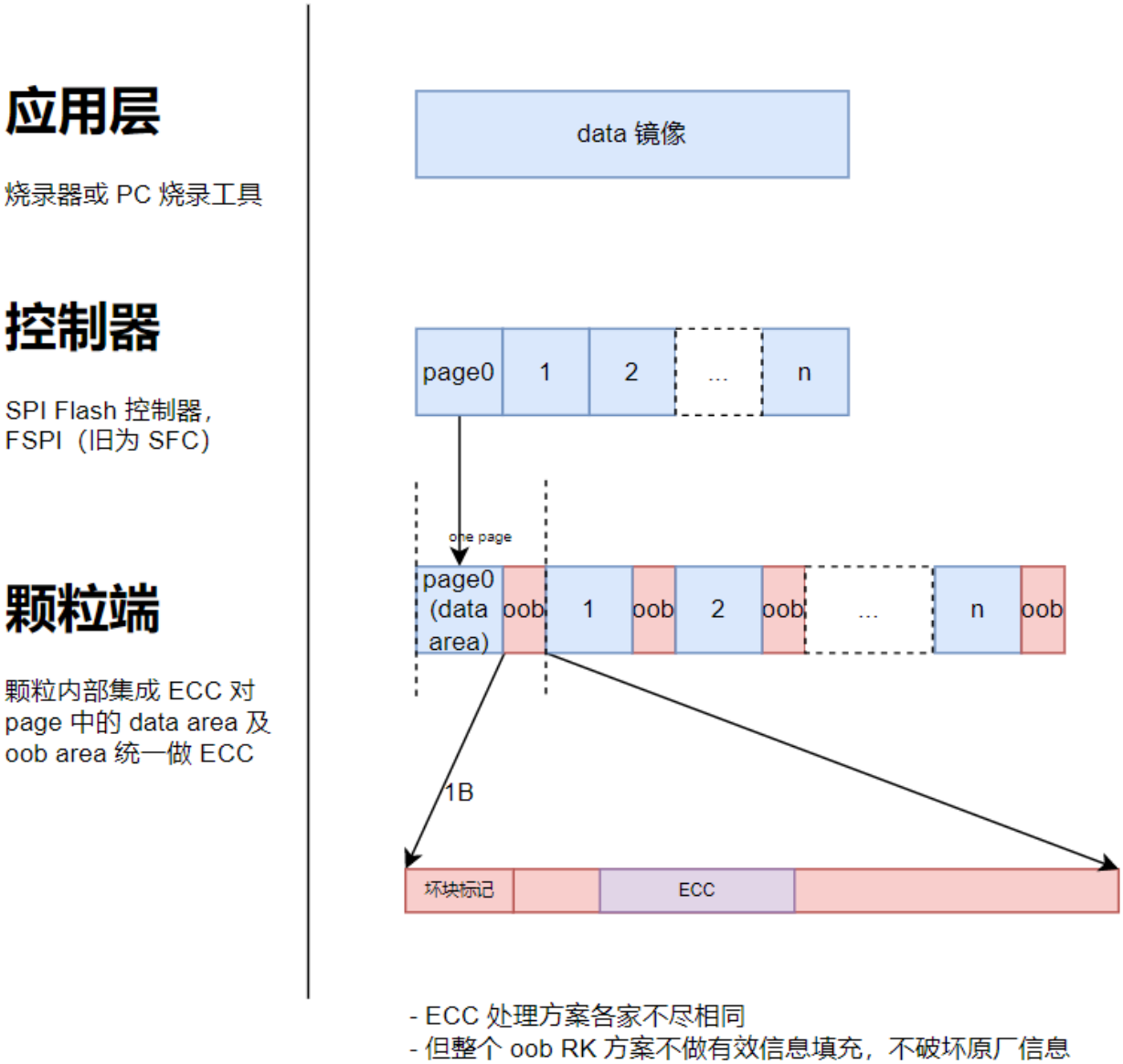
Burn the A standalone image at address 0, and burn the B standalone image at address 8MB, which means writing skip bad block to burn the image from the partition start

The software's strategy for detecting partition images, reading skip bad block to load data from the partition start

The burning strategy and software strategy are consistent.

8.7 SPI Nand SDK Outputs Valid Data Image Without Redundant OOB Data

Main Reason: The SPI Flash controller does not integrate ECC, hence it relies on the ECC module at the memory device.



8.8 SPI Nand rkflash Kernel Phase Read/Write Speed Optimization

When there is a large amount of IO data, there is indeed a higher CPU usage rate, but it can be improved by sacrificing some flash speed to enhance CPU scheduling.

```
diff --git a/drivers/rkflash/sfc_nand.c b/drivers/rkflash/sfc_nand.c
index 0bc1c6b43fd7..ca190c115ad3 100644
--- a/drivers/rkflash/sfc_nand.c
+++ b/drivers/rkflash/sfc_nand.c
@@ -274,7 +274,7 @@ static int sfc_nand_wait_busy(u8 *data, int timeout)

        *data = 0;

-       for (i = 0; i < timeout; i++) {
+       for (i = 0; i < timeout; i += 20) {
                ret = sfc_nand_read_feature(0xC0, &status);

                if (ret != SFC_OK)
@@ -285,7 +285,7 @@ static int sfc_nand_wait_busy(u8 *data, int timeout)
                if (!(status & (1 << 0)))
                        return SFC_OK;

-               sfc_delay(1);
+               usleep_range(20, 30);
        }

        return SFC_NAND_WAIT_TIME_OUT;
```

8.9 SPI Nand mtd Solution Kernel Phase Flash Information and Bad Block Information

Bad Block Information:

```
cat /sys/devices/virtual/mtd/mtd5/bad_blocks
```

Other Information:

```
# ls /sys/devices/virtual/mtd/mtd4/
bad_blocks         ecc_step_size      numeraseregions    subsystem/
bbt_blocks         ecc_strength       offset             type
bitflip_threshold  erasesize          oobsize            uevent
corrected_bits     flags              power/             writesize
dev                mtdblock4/         size
ecc_failures       name               subpagesize
```

8.10 SPI Nand mtd Solution Kernel Phase mtd Write Failure

Confirm if the following memory devices are used:

- S35ML01G3, ANV1GCP0CLG, HYF1GQ4UTXCAE, YX25G1E
- S35ML02G3, ANV2GCP0CLG, HYF2GQ4UTXCAE, YX25G2E

The compatibility issue with the 84h command for these memory devices has been resolved with a patch, and a correction to the original manufacturer documentation has been requested.

```
commit 328145662f6d6154fbf4329a0d53f9c152673648
Author: Jon Lin <jon.lin@rockchip.com>
Date:   Fri Jul 7 12:06:40 2023 +0800

    mtd: spinand: skyhigh: The vendor requires the devices to be patched

    1.Double OIP=0 after page 13H
    2.The nand flash does not support 84H and 34H command

    Change-Id: Ie805f42a36e1a864115988087bdc43592cc94ded
    Signed-off-by: Jon Lin <jon.lin@rockchip.com>
```

8.11 SPI Nand mtd Solution Root Filesystem Auto-mount Compatibility with eMMC Solution

By default, the SDK does not support this feature. If compatibility with eMMC is required, it is recommended to identify the type of memory device as either eMMC or spi-nand in uboot, and then pass the root parameters to the kernel's bootargs.

8.12 SPI Nand mtd Solution for env Partition Table Upgrade Compatibility with 4K page-size NAND devices using idb Image

Memory devices with 4KB page-size require a special command to generate the idblock image for upgrade purposes. Command reference:

```
./tools/programmer_image_tool -i rv1106_download_v1.14.108.bin -b 128 -p 4 -2 -t spinand -o
./
```

Description:

- Outputs the idblock.img target image
- The latest SDK defaults to outputting this image.

8.13 SPI Nand Fast Boot Solution Kernel Failure to Establish Bad Block Table

SPI Nand page read rate anomalies and optimization, the fast boot solution without passing through uboot, the establishment of the bad block table during the kernel phase has compatibility issues and is not established.

```
commit c7270465b2a699211dfd8ad3c2f882f22247c40c
Author: Jon Lin <jon.lin@rock-chips.com>
Date:   Thu Dec 17 16:05:27 2020 +0800

    drivers: rkflash: Fixed bbt operation calculation error

    Fixes: 008340a82 ("drivers: rkflash: Change to use the api which the oob area available")
    Change-Id: I140aa76a2acb73271ba04b7060030dc06a2353e6
    Signed-off-by: Jon Lin <jon.lin@rock-chips.com>
```

8.14 SPI Nand Custom MTD Upgrade Solution Encounters Anomalies with Bad Block Partitions

Source of the Problem:

Typically, random address writing behavior exists within the partition, for example: An image of about 2M in size, written in units of 512K using nandwrite, each time nandwrite is offset by 512K for the next write. If a bad block is encountered during this process, it will cause the uboot mtd read to fail.

Recommendation:

It is required to write in one go; otherwise, you must manage bad blocks on your own.

Reason for the One-Time Write Requirement:

If the partition header is not written in one go, there is a possibility of not being able to effectively recognize bad blocks. Assuming the bad block is 8:

- Write 7_8, actually write 7_9
- Write 9, actually write 9, so 9 will be written repeatedly, causing the data in 9 to be read back with ECC failure.

8.15 SPI Nand Continuous Read Support and Quick Boot Adaptation

8.15.1 Introduction

SPI Nand manufacturer Winbond has implemented special support for fast loading of SPI Nand and has applied it to specific SPI Nand physics, taking W25N01GV as an example, supporting:

- Support for bad block substitution, where after software setting, a specific area has no bad blocks
- Support for continuous read, where clock data continues to output beyond the page boundary and sustains throughput for the next space of data
- Support for SDR 104MHz continuous read clock rate, capable of supporting SDR mode 52MB/s throughput

Winbond W25N01XX SPI Nand CONT_READ quick start solution, referred to as the SPI Nand quick start scheme, based on the CONT_READ function of related materials, achieves:

- Support for ROM phase CONT_READ acceleration
- Support for SPL phase CONT_READ acceleration
- Support for SPL phase specific firmware FSPI DMA background asynchronous loading implementation

8.15.2 Configuration

rkbin ini file example with RV1103B: RV1103BMINIALL_SPI_NAND_W25N01XX_TB.ini

u-boot red configuration:

```
CONFIG_SPI_NAND_WINBOND_CONT_READ
```

9. SPI NOR Issues

9.1 SPI Nor Flash 1V8 256Mbits and Higher Capacity Material Selection

Date	Vendor	Model	Density (Gbit)	Capacity (MB)	Voltage	Status
--	MXIC	MX25U25645GZ4I-00	256MBits	32MB	1.8V	Supported
--	MXIC	MX25U51245GZ4I00	512Mbits	64MB	1.8V	Supported
--	GigaDevice	GD25LQ256CW1G	256MBits	32MB	1.8V	Supported
--	Winbond	W25Q256JWEQ	256MBits	32MB	1.8V	Supported
--	XTX	XT25W512BSFHGU	512Mbit	--	1.65~3.6V	Not Verified
--	Dosilicon	DS25M4BA-1AIB1	256Mb	--	--	Not Verified
--	Dosilicon	DS25M4BA-1AIB4	256Mb	--	--	Not Verified

You can communicate with these manufacturers to see if they have any parts that meet customer requirements, including RK not verified parts that can provide samples for verification. The supported parts list should be based on the latest flash support list.

9.2 SPI Nor Flash Solutions Compared with Kernel 4.4 and Earlier Versions of the MTD Open Source Framework

The older kernel open source framework supports a limited number of flash devices and has an older MTD framework structure, which is why RK did not adapt it. In contrast, the rkflash storage solution implements and is compatible with a wider range of SPI flash devices.

9.3 SPI Nor mtd Solution u-boot Down sf Command and Related Interface Implementation

RK SDK u-boot uses the DM_SPI_FLASH framework, supporting the following interfaces under spi_flash.h:

- Initialization interfaces such as spi_find_bus_and_cs and spi_flash_probe_bus_cs
- Read and write interfaces like spi_flash_read

For how to use the related interfaces, it is recommended to refer to the source code of cmd/sf.c and u-boot commands.

cmd_sf

Taking RV1106 as an example:

Patch:

```
diff --git a/arch/arm/dts/rv1106-u-boot.dtsi b/arch/arm/dts/rv1106-u-boot.dtsi
index b0632e9ce9..03bc933125 100644
--- a/arch/arm/dts/rv1106-u-boot.dtsi
+++ b/arch/arm/dts/rv1106-u-boot.dtsi
@@ -167,6 +167,7 @@
                spi-tx-bus-width = <1>;
                spi-rx-bus-width = <4>;
                spi-max-frequency = <80000000>;
+               status = "disabled";
        };

        spi_nor: flash@1 {
diff --git a/configs/rv1106_defconfig b/configs/rv1106_defconfig
index 9b20bcfb65..b1f3933a11 100644
--- a/configs/rv1106_defconfig
+++ b/configs/rv1106_defconfig
@@ -148,3 +148,4 @@ CONFIG_SPL_LZMA=y
 CONFIG_SPL_GZIP=y
 CONFIG_ERRNO_STR=y
 # CONFIG_EFI_LOADER is not set
+CONFIG_CMD_SF=y
```

Command:

```
=> sf probe 2:0 10000000 3
SF: Detected sfc_nor with page size 256 Bytes, erase size 4 KiB, total 16 MiB

=> sf read 0x400000 0x200 0x200
device 0 offset 0x200, size 0x200
SF: 512 bytes @ 0x200 Read: OK
```

9.4 SPI Nor mtd Registration of mtdparts Takes Long Time

Each mtd partition is an mtd device, and if mtd block is enabled, block devices must also be registered, making the overall registration time relatively long:

add_mtd_device:

- device_register 0.15ms
- mtd_nvmem_add 0.08ms
- device_create 0.12ms
- mtdpstore_notify_add 0.5ms

9.5 SPI Nor mtd Solution: Slow Erasure Speed in U-Boot

If the JFFS2 file system is not used in the U-Boot environment, the macro SPI_FLASH_USE_4K_SECTORS can be removed, and 64KB alignment erasure can be used instead.

9.6 SPI Nor Single-Line and Quad-Line Transfer Performance Differences

Testing Environment: IO Clock 80MHz.

IO Utilization	io0~3 Bi-Directional	io0~1 Bi-Directional	io0 Receive, io1 Transmit
File System Read	35MB/s	19MB/s	11MB/s
File System Erase Write	150KB/s	150KB/s	150KB/s

9.7 SPI Nor High-Speed Copying Anomaly

Taking a nominal 104 MHz SPI Nor device as an example

- The SPI Nor device is nominally 104MHz, but a small number of characteristics are constrained at 83MHz.
- Testing this SPI Nor device under 80MHz conditions is unstable.

Test Objective

- Pass the copying test at an 80 MHz IO rate.

Signal Quality Confirmation

It is recommended to first confirm the signal quality:

- Key signals during the operation of SPI Nor in the testing kernel phase, including clk, d0, d1, cs, should ideally be captured simultaneously to confirm if there are any obvious deficiencies in the signal quality during the kernel phase.

Testing

If there are no obvious signal issues, it is suggested to:

- Test A: Reduce the frequency of the problematic device to 60MHz and perform some high and low temperature auto reboot tests to see if it can pass under stress.
- Test B: Choose another original device with a nominal 100MHz and perform auto reboot tests as a control to prove whether there is a quality issue with the board itself.
- Based on the test results, determine if 60M is stable:
    - If both A and B fail, it is highly likely that there is a hardware issue with the board, preventing the SPI Nor rate from increasing, further confirming the signal.
    - If both A and B pass, considering the default firmware fails at 80MHz, it is more likely that the problematic device is falsely marked.
    - If A fails and B passes, continue to reduce the frequency of A to 40M and continue testing. If 40MHz passes, then A is severely falsely marked or has extremely poor compatibility.

Further Processing Based on Test Results

- If the test is relatively good and it is determined to use this device, then evaluate whether 60MHz meets the product performance requirements.

9.8 SPI Nor AUTO_MERGE Technology

9.8.1 Introduction

Due to the market situation in the SPI Nor flash market where "the price of 2 pieces of 16MB SPI Nor flash is significantly advantageous compared to 1 piece of 32MB SPI Nor flash," there is a scenario where 2 SPI Nor flashes are used together. The RK SPI Nor AUTO_MERGE software technology merges 2 SPI Nor flashes into 1 large-capacity SPI Nor flash, thereby gaining convenience in solution implementation, development, and maintenance:

- Device tree configuration for 1 Nor device
- Framework layer scans 1 AUTO_MERGE Nor device with 2 sub-devices
- User layer sees 1 Nor device

9.8.2 Limitations

- The AUTO_MERGE software solution only supports the combination of two identical SPI Nor devices and requires similar PCB layouts.
- The AUTO_MERGE software solution is compatible with single SPI Nor devices, but the PIN corresponding to CS1N GPIO will have signal output during the SPI Nor detection process.
- The SPL FSPI DMA asynchronous background loading area must not span across SPI Nor devices.
    - For example: In the fast startup project, the ramdisk will enable the preload attribute in the dts.
- The open-source usbplug supports self-compilation and is only compatible with USB downloads.
    - For UART downloads, please use the closed-source download_xxxcs1.bin, usbplug_xxxcs1.bin (self-packaged).

9.8.3 Configuration

rkbin ini file example with RV1103B: RKBOOT/RV1103BMINIALL_AUTO_MERGE.ini

usbplug/spl/u-boot macro switch:

```
CONFIG_SPI_FLASH_AUTO_MERGE=y
CONFIG_SPL_GPIO_SUPPORT=y
```

usbplug/spl/u-boot dts switch:

```
diff --git a/arch/arm/dts/rv1103b-u-boot.dtsi b/arch/arm/dts/rv1103b-u-boot.dtsi
index a31355a..88ceb85 100644
--- a/arch/arm/dts/rv1103b-u-boot.dtsi
+++ b/arch/arm/dts/rv1103b-u-boot.dtsi
@@ -67,12 +67,12 @@
 };

 &gpio1 {
-       u-boot,dm-pre-reloc;
+       u-boot,dm-spl;
        status = "okay";
 };

 &gpio2 {
-       u-boot,dm-pre-reloc;
+       u-boot,dm-spl;
        status = "okay";
 };

@@ -92,6 +92,7 @@
 };

 &sfc {
+       sfc-cs-gpios= <&gpio1 RK_PA5 GPIO_ACTIVE_LOW>, <&gpio2 RK_PA6 GPIO_ACTIVE_LOW>;
        u-boot,dm-spl;
        status = "okay";
        };
```

kernel macro switch:

```
CONFIG_MTD_SPI_NOR_AUTO_MERGE=y
```

kernel dts reference:

```
&fspi0 {
        sfc-cs-gpios = <&gpio1 RK_PA5 GPIO_ACTIVE_LOW>, <&gpio1 RK_PA6 GPIO_ACTIVE_LOW>;
        pinctrl-names = "default";
        pinctrl-0 = <&fspi_cs0n &fspi_cs1n>;
        status = "okay";

        flash@0 {
                compatible = "jedec,spi-nor";
```

```
                reg = <0>;
                spi-max-frequency = <75000000>;
                spi-rx-bus-width = <4>;
                spi-tx-bus-width = <1>;
        };
};

&pinctrl {
        fspi {
                fspi_cs0n: fspi-cs0n {
                        rockchip,pins = <1 RK_PA5 RK_FUNC_GPIO &pcfg_pull_up>;
                };
                fspi_cs1n: fspi-cs1n {
                        rockchip,pins = <1 RK_PA6 RK_FUNC_GPIO &pcfg_pull_up>;
                };
        };
};
```

9.8.4 CSN1 Configuration

It is recommended to use the RK-recommended CSN1 GPIO pin, and the hardware schematics should be obtained by contacting the relevant hardware engineer.

USB plug/spl/uboot dts configuration, taking the modification of RV1103B GPIO1A6 to GPIO1A7 as an example:

```
diff --git a/arch/arm/dts/rv1103b-u-boot.dtsi b/arch/arm/dts/rv1103b-u-boot.dtsi
index d346eae20bf..86265f76304 100644
--- a/arch/arm/dts/rv1103b-u-boot.dtsi
+++ b/arch/arm/dts/rv1103b-u-boot.dtsi
@@ -92,7 +92,7 @@
 };

 &sfc {
-       sfc-cs-gpios= <&gpio1 RK_PA5 GPIO_ACTIVE_LOW>, <&gpio1 RK_PA6 GPIO_ACTIVE_LOW>;
+       sfc-cs-gpios= <&gpio1 RK_PA5 GPIO_ACTIVE_LOW>, <&gpio1 RK_PA7 GPIO_ACTIVE_LOW>;
        u-boot,dm-spl;
        status = "okay";

diff --git a/arch/arm/mach-rockchip/rv1103b/rv1103b.c b/arch/arm/mach-
rockchip/rv1103b/rv1103b.c
index 7353528afdd..8e30240076e 100644
--- a/arch/arm/mach-rockchip/rv1103b/rv1103b.c
+++ b/arch/arm/mach-rockchip/rv1103b/rv1103b.c
@@ -252,9 +252,9 @@ int arch_cpu_init(void)
        writel(0x00000000, USBPHY_APB_BASE + USBPHY_FSLS_DIFF_RECEIVER);

 #ifdef CONFIG_SPI_FLASH_AUTO_MERGE
-       /* gpio1a5/gpio1a6 cs-gpio */
+       /* gpio1a5/gpio1a7 cs-gpio */
        writel(0x00F00000, GPIO1_IOC_BASE + GPIO1A_IOMUX_SEL_1_0);
```

```
-       writel(0x0F000000, GPIO1_IOC_BASE + GPIO1A_IOMUX_SEL_1_1);
+       writel(0xF0000000, GPIO1_IOC_BASE + GPIO1A_IOMUX_SEL_1_1);
 #endif
 #endif
```

Kernel configuration, taking the modification of RV1103B GPIO1A6 to GPIO1A7 as an example:

```
diff --git a/arch/arm/boot/dts/rv1103b-evb1-v10-spi-nor-auto-merge.dts
b/arch/arm/boot/dts/rv1103b-evb1-v10-spi-nor-auto-merge.dts
index ae3fa579b05f..00bf8008e107 100644
--- a/arch/arm/boot/dts/rv1103b-evb1-v10-spi-nor-auto-merge.dts
+++ b/arch/arm/boot/dts/rv1103b-evb1-v10-spi-nor-auto-merge.dts
@@ -14,7 +14,7 @@ / {
 };

 &fspi0 {
-       sfc-cs-gpios = <&gpio1 RK_PA5 GPIO_ACTIVE_LOW>, <&gpio1 RK_PA6 GPIO_ACTIVE_LOW>;
+       sfc-cs-gpios = <&gpio1 RK_PA5 GPIO_ACTIVE_LOW>, <&gpio1 RK_PA7 GPIO_ACTIVE_LOW>;
        pinctrl-names = "default";
        pinctrl-0 = <&fspi_cs0n &fspi_cs1n>;
        status = "okay";
@@ -34,7 +34,7 @@ fspi_cs0n: fspi-cs0n {
                        rockchip,pins = <1 RK_PA5 RK_FUNC_GPIO &pcfg_pull_up>;
                };
                fspi_cs1n: fspi-cs1n {
-                       rockchip,pins = <1 RK_PA6 RK_FUNC_GPIO &pcfg_pull_up>;
+                       rockchip,pins = <1 RK_PA7 RK_FUNC_GPIO &pcfg_pull_up>;
                };
        };
 };
```

10. Tool Issues

10.1 Creating GPT Image with upgrade_tool

Utilize the Linux upgrade tool, version v1.67 or higher.

```
./upgrade_tool gpt parameter.txt  gpt.img
```

10.2 Boot Merger Unpack Loader

```
./tools/boot_merger unpack -i rk356x_spl_loader_v1.01.102-528M-1115.bin -o .
```

10.3 Android Upgrade Utility LoaderToDDR Features

There are two configuration methods for the loader item on the tool:

1. loader —— The function for initiating the upgrade and extracting the idb.img for upgrade from the loader image.
2. loaderToDDR —— The function solely for initiating the upgrade.

10.4 Introduction to the idb_bootconfig Tool

The romcode by default loads the firmware in SPI Nor using a single-line transmission method. To accelerate the modification to four-line loading of firmware, the idb packaging image header can be enhanced with a bootconfig. The idb_bootconfig script is a tool designed to add NOR flash startup acceleration configurations within the idb image.

Due to the varying bootconfig parameters for different SPI Nor flash configurations, it is recommended to add them within the idb_bootconfig rather than pre-filling data in the image. Otherwise, a specific bootconfig parameter would need to be tailored for each SPI Nor flash variant.